

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA ĐIỆN – ĐIỆN TỬ
BỘ MÔN ĐIỆN TỬ
-----o0o-----



ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC

**Thiết kế hệ thống giám sát và cảnh báo
thông số môi trường đa điểm cho nhà kho
ứng dụng mạng Mesh, 4G LTE và MQTT**

GVHD: ThS. Nguyễn Trọng Luật

SVTH: Nguyễn Trường Sơn

MSSV: 2212930

TP. HỒ CHÍ MINH, THÁNG 5 NĂM 2026

KHOA ĐIỆN – ĐIỆN TỬ		NHIỆM VỤ KHÓA LUẬN TỐT NGHIỆP	
		(Chú ý: sinh viên phải dán tờ này vào trang thứ nhất của bản thuyết minh)	
HỌ VÀ TÊN:	NGUYỄN TRƯỜNG SƠN	MSSV:	2212930
NGÀNH:	ĐIỆN TỬ - VIỄN THÔNG	LỚP:	DD22DVT3
1. Đầu đề khóa luận:			
Thiết kế hệ thống giám sát và cảnh báo thông số môi trường đa điểm cho nhà kho ứng dụng mạng Mesh, 4G LTE và MQTT			
Designing a multi-point environmental monitoring and alerting system for a warehouse using Mesh network, 4G LTE, and MQTT			
2. Nhiệm vụ (yêu cầu về nội dung và số liệu ban đầu):			
-	Xây dựng hệ thống truyền dữ liệu từ trạm này qua trạm khác gồm các Node và 1 Gateway sử dụng công nghệ Mesh		
-	Thu thập dữ liệu môi trường và gửi dữ liệu lên server để giám sát và phát tín hiệu cảnh báo		
-	Xây dựng website hiển thị các dữ liệu để giám sát dữ liệu		
3. Ngày giao nhiệm vụ khóa luận:		01 / 01 / 2026	
4. Ngày hoàn thành nhiệm vụ:		11 / 05 / 2026	
5. Họ và tên người hướng dẫn:		Phản hướng dẫn:	
-	Nguyễn Trọng Luật	Xây dựng hệ thống truyền dữ liệu từ trạm này qua trạm khác gồm các Node và 1 Gateway sử dụng công nghệ Mesh	
Nội dung và yêu cầu KLTN đã được thông qua Chủ trì ngành.			

Ngàytháng năm

CHỦ TRÌ NGÀNH

(Ký và ghi rõ họ tên)

GIẢNG VIÊN HƯỚNG DẪN CHÍNH

(Ký và ghi rõ họ tên)

PHẦN DÀNH CHO KHOA

Người duyệt (chấm sơ bộ):

Đơn vị:

Ngày bảo vệ:

Điểm tổng kết:

Nơi lưu trữ luận án:

LỜI CẢM ƠN

Em xin trân trọng gửi lời cảm ơn đến thầy Nguyễn Trọng Luật, giảng viên hướng dẫn đồ án tốt nghiệp của em, vì đã luôn tận tình chỉ bảo, định hướng và góp ý chi tiết trong suốt quá trình thực hiện. Những góp ý chuyên môn cùng sự động viên của thầy đã giúp em xác định rõ mục tiêu, lựa chọn phương pháp phù hợp và hoàn thiện sản phẩm theo đúng tiến độ.

Em cũng xin gửi lời cảm ơn đến quý thầy cô trong Khoa đã truyền đạt cho em nền tảng kiến thức vững chắc trong suốt thời gian học tập tại trường. Chính những kiến thức cơ sở và kỹ năng tư duy được tích lũy từ các học phần đã giúp em triển khai ý tưởng, xử lý các vấn đề kỹ thuật phát sinh và trình bày kết quả một cách mạch lạc, có định hướng.

Bên cạnh đó, em chân thành cảm ơn gia đình đã luôn ủng hộ, tạo điều kiện về thời gian và tinh thần; cảm ơn thầy cô, anh chị kỹ thuật và các bạn trong lớp/nhóm đã hỗ trợ, chia sẻ tài liệu, góp ý và cùng em thảo luận trong quá trình làm đồ án. Những hỗ trợ quý báu ấy là động lực để em nỗ lực hoàn thành đề tài.

Một lần nữa, em xin chân thành cảm ơn!

Tp. Hồ Chí Minh, ngày 11 tháng 5 năm 2026.

Sinh viên



Nguyễn Trường Sơn

TÓM TẮT LUẬN VĂN

Đồ án này tập trung vào nghiên cứu và triển khai một hệ thống giám sát môi trường đa điểm thông minh dựa trên kiến trúc mạng ESP-MESH. Hệ thống bao gồm các Leaf node chịu trách nhiệm thu thập thông số cảm biến, hai Relay node đóng vai trò trung gian chuyển tiếp đa chặng, và một Root node cố định vai trò MESH-ROOT đóng vai trò là Gateway trung tâm kết nối với MQTT Broker trong mạng LAN. Hệ thống được thiết kế nhằm mục tiêu đảm bảo truyền dữ liệu ổn định, mở rộng vùng phủ sóng linh hoạt mà không cần tăng công suất phát của từng nút đơn lẻ. Bên cạnh đó, hệ thống còn được trang bị Dashboard Web thời gian thực, cho phép người dùng giám sát trực quan các thông số thông qua biểu đồ và thực hiện cấu hình ngưỡng cảnh báo từ xa một cách linh hoạt.

Để thực hiện đề tài này, em đã nghiên cứu các phần cứng và phần mềm để hệ thống hoạt động ổn định. Em quyết định sử dụng vi điều khiển ESP32 làm các Node và cũng như là gateway cho hệ thống. Nhiệm vụ nhằm thu thập dữ liệu môi trường và gửi dữ liệu lên hệ thống. Đặc biệt, tại Server, hệ thống tích hợp module 4G LTE A7680C để thực hiện các kịch bản cảnh báo khẩn cấp qua cuộc gọi VoLTE và tin nhắn SMS trực tiếp đến số điện thoại người quản lý ngay khi phát hiện thông số vượt ngưỡng. Các Node và Gateway trao đổi dữ liệu với nhau bằng công nghệ Mesh nên có thể mở rộng được vùng phủ sóng trong môi trường nhà kho quá rộng.

Sau một học kì thực hiện đề tài, em đã thiết kế được một hệ thống có thể thu thập được các dữ liệu nhiệt độ, độ ẩm và ánh sáng có mức sai số trung bình trong khoảng 5%. Các chức năng cảnh báo như cảnh báo có người cũng đáp ứng rất nhanh và cảnh báo gửi tin nhắn và gọi điện về điện thoại người dùng trong khoảng 10 giây. Khoảng cách kết nối giữa các node với nhau là 25m trong khi đó một Router cơ bản có vùng phủ sóng từ 30-50m. Hệ thống cơ bản có thể đáp ứng đủ các yêu cầu mà em đặt ra và có thể áp dụng vào thực tế trong môi trường nhà kho.

MỤC LỤC

1. GIỚI THIỆU	1
1.1 Tổng quan.....	1
1.2 Tình hình nghiên cứu trong và ngoài nước	1
1.3 Nhiệm vụ đề tài	1
2. LÝ THUYẾT	2
2.1 Giới thiệu tổng quan về ESP MESH.....	2
2.2 Giới thiệu về giao thức MQTT.....	6
2.3 Tìm hiểu về các tập lệnh AT	7
2.4 Tìm hiểu về các linh kiện trong hệ thống.....	8
2.4.1 ESP32 Devkit V1	8
2.4.2 Cảm biến nhiệt độ và độ ẩm DHT11	9
2.4.3 Cảm biến ánh sáng BH1750 GY-302.....	9
2.4.4 Cảm biến phát hiện chuyển động PIR	10
2.4.5 Module 4G LTE A7680C.....	11
2.4.6 Module màn hình OLED V1 0.96 inch	12
2.4.7 Module chuyển đổi USB qua TTL CP2102	13
2.5 Tổng quan về Website.....	14
2.5.1 Giới thiệu tổng quan về Website	14
2.5.2 Ngôn ngữ HTML.....	15
2.5.3 Ngôn ngữ CSS.....	16
2.5.4 Ngôn ngữ Javascript.....	16
2.5.5 Database MongoDB	17
3. TỔNG QUAN VỀ HỆ THỐNG.....	18
4. THIẾT KẾ VÀ THỰC HIỆN PHẦN CỨNG	20
4.1 Yêu cầu thiết kế.....	20
4.2 Lựa chọn linh kiện và phân tích	20

4.3	Sơ đồ khối tổng quát	26
4.4	Thiết kế mạch chi tiết	27
4.4.1	Thiết kế mạch cho Leaf Node	27
4.4.2	Thiết kế mạch cho Relay Node	28
4.4.3	Thiết kế mạch cho Root Node	29
4.4.4	Module Sim cảnh báo từ Server	29
5.	THIẾT KẾ VÀ THỰC HIỆN PHẦN MỀM	30
5.1	Lưu đồ giải thuật của Leaf Node	30
5.2	Lưu đồ giải thuật của Relay Node	31
5.3	Lưu đồ giải thuật của Root Node	33
5.4	Sơ đồ khối của Dashboard	34
6.	KẾT QUẢ THỰC HIỆN	36
6.1	Kết quả thực hiện hệ thống	36
6.2	Kết quả kiểm tra hệ thống	48
6.2.1	Đo đặc số liệu cảm biến DHT11	48
6.2.2	Đo đặc số liệu cảm biến BH1750 GY-302	52
7.	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	55
7.1	Kết luận	55
7.2	Hướng phát triển	55
8.	TÀI LIỆU THAM KHẢO	56
9.	PHỤ LỤC	57

DANH SÁCH HÌNH MINH HỌA

Hình 2.1: Mô hình ESP-MESH	2
Hình 2.2: Cấu trúc cây trong ESP-MESH	3
Hình 2.3: ESP-MESH với cơ chế Station–SoftAP đồng thời	4
Hình 2.4: Cơ chế tự phục hồi của ESP-MESH	5
Hình 2.5: ESP-MESH software stack	6
Hình 2.6: Giao thức MQTT	7
Hình 2.7: Sơ đồ chân của ESP32 Devkit V1	8
Hình 2.8: Cảm biến DHT11	9
Hình 2.9: Cảm biến BH1750 GY-302.....	10
Hình 2.10: Cảm biến PIR.....	11
Hình 2.11: Module A7680C	12
Hình 2.12: Màn hình OLED 0.96 inch	13
Hình 2.13: Module chuyển đổi USB qua TTL CP2102.....	13
Hình 2.14: Database MongoDB.....	18
Hình 3.1: Sơ đồ tổng quát hệ thống.....	19
Hình 4.1: Block Diagram	26
Hình 4.2: Schematic của Leaf Node.....	27
Hình 4.3: Leaf Node thực tế	28
Hình 4.4: Relay Node thực tế	28
Hình 4.5: Root Node thực tế	29
Hình 4.6: Module SIM ở server	29
Hình 5.1: Lưu đồ giải thuật của Leaf Node.....	30
Hình 5.2: Lưu đồ giải thuật của Relay Node.....	31
Hình 5.3: Lưu đồ giải thuật của Root Node.....	33
Hình 5.4: Sơ đồ khối của Dashboard	34
Hình 6.1: Leaf Node chọn Parent Node.....	36

Hình 6.2: Leaf Node gửi dữ liệu	36
Hình 6.3: Dữ liệu hiển thị trên OLED	37
Hình 6.4: Địa chỉ STA và SoftAP của Relay Node A.....	37
Hình 6.5: Relay Node A đã kết nối được với Root Node	37
Hình 6.6: Địa chỉ STA và SoftAP của Relay Node B.....	37
Hình 6.7: Relay Node B đã kết nối được với Root Node	37
Hình 6.8: Địa chỉ STA và SoftAP của Root Node	38
Hình 6.9: Các Child Node đã kết nối được đến Root Node	38
Hình 6.10: Root Node đã nhận được dữ liệu từ Leaf Node	38
Hình 6.11: Topic chứa dữ liệu cảm biến.....	39
Hình 6.12: Topic chứa báo cáo trạng thái mạng	39
Hình 6.13: Dashboard của hệ thống.....	40
Hình 6.14: Dữ liệu lưu trữ tại Mongo DB	41
Hình 6.15: 2 Leaf Node gửi dữ liệu.....	42
Hình 6.16: Hệ thống hoạt động bình thường.....	42
Hình 6.17: Hệ thống gặp sự cố	43
Hình 6.18: Dashboard hiển thị hệ thống gặp sự cố mất node	43
Hình 6.19: Chức năng tự phục hồi của ESP-Mesh.....	44
Hình 6.20: Dashboard hiển thị hệ thống đã thực hiện chức năng tự phục hồi	44
Hình 6.21: Quá trình mạng Mesh thực hiện chức năng tự phục hồi	44
Hình 6.22: Chức năng điều chỉnh ngưỡng.....	45
Hình 6.23: Hệ thống phát hiện vượt ngưỡng.....	46
Hình 6.24: Thực hiện hàm cảnh báo	46
Hình 6.25: Nhận cảnh báo bằng tin nhắn SMS ở điện thoại	47
Hình 6.26: Đăng ký thông tin khách hàng lên database.....	48
Hình 6.27: Thiết bị đo Electronic Hygrometer	49
Hình 6.28: Kết quả so sánh nhiệt độ giữa cảm biến và thiết bị	50

Hình 6.29: Kết quả so sánh độ ẩm giữa cảm biến và thiết bị.....	50
Hình 6.30: Thiết bị đo Luxmeter.....	52
Hình 6.31: Kết quả so sánh giá trị ánh sáng giữa cảm biến và thiết bị	53

DANH SÁCH BẢNG SỐ LIỆU

Bảng 2.1: Đánh giá chất lượng kết nối theo RSSI	5
Bảng 4.1: Phương án chọn vi điều khiển cho hệ thống	20
Bảng 4.2: Phương án chọn cảm biến nhiệt độ cho hệ thống.....	21
Bảng 4.3: Phương án chọn cảm biến ánh sáng cho hệ thống	22
Bảng 4.4: Phương án chọn cảm biến phát hiện chuyển động cho hệ thống	23
Bảng 4.5: Phương án chọn thiết bị hiển thị cho hệ thống	24
Bảng 4.6: Phương án chọn module Sim cho hệ thống.....	25
Bảng 6.1: Giá trị đo từ cảm biến DHT11 so với thiết bị.....	49
Bảng 6.2: Giá trị sai số của giá trị đo từ cảm biến DHT11 so với thiết bị	51
Bảng 6.3: Giá trị đo từ cảm biến BH1750 GY-302 so với thiết bị	52
Bảng 6.4: Giá trị sai số của giá trị đo từ cảm biến BH1750 GY-302 so với thiết bị.....	54

1. GIỚI THIỆU

1.1 Tổng quan

Đồ án “Thiết kế hệ thống giám sát và cảnh báo thông số môi trường đa điểm cho nhà kho ứng dụng mạng Mesh, 4G LTE và MQTT” là đề tài nghiên cứu trong lĩnh vực IoT về truyền dữ liệu. Đề tài tập trung nghiên cứu và xây dựng hệ thống giám sát môi trường đa điểm ứng dụng công nghệ ESP-MESH để mở rộng được khoảng cách kết nối ngoài vùng phủ sóng của Router kết hợp với hạ tầng viễn thông 4G LTE đảm bảo khả năng cảnh báo qua cuộc gọi và tin nhắn chất lượng cao. Hệ thống giải quyết bài toán mở rộng vùng phủ sóng không dây linh hoạt và quản lý tập trung trên Dashboard , giúp người dùng giám sát trực quan và điều khiển các ngưỡng cảnh báo từ xa một cách hiệu quả.

Mục tiêu của đề tài là thu thập các giá trị môi trường trong môi trường nhà kho từ các cảm biến và truyền các dữ liệu qua các Node thông qua công nghệ Mesh. Từ đó gửi các dữ liệu thu được lên server để giám sát qua website và cảnh báo vượt ngưỡng an toàn.

1.2 Tình hình nghiên cứu trong và ngoài nước

Sau khi tìm hiểu các bài báo nghiên cứu liên quan đến đề tài. Em nhận thấy các nghiên cứu về hệ thống truyền dữ liệu sử dụng Mesh như hệ thống giám sát nông nghiệp. Hoặc các hệ thống sử dụng các phương pháp truyền dữ liệu khác. Các hệ thống này chỉ thực hiện cảnh báo dữ liệu lên Website chứ chưa có đầy đủ thông tin đến người dùng và người dùng tại hiện trường cũng không thể theo dõi trực tiếp được mà phải lên Website để xem dữ liệu. Vì vậy từ những tìm hiểu và kết quả nghiên cứu trước đó, em muốn cải tiến thêm về chức năng cảnh báo trực tiếp đến người dùng và người dùng tại hiện trường có thể theo dõi dữ liệu nhanh chóng.

1.3 Nhiệm vụ đề tài

Để hiện thực hóa hệ thống, đề tài tập trung thực hiện các nhiệm vụ trọng tâm sau:

Nhiệm vụ 1: Thiết kế mạng Mesh nội bộ thông qua việc triển khai kiến trúc mạng ESP-MESH bao gồm các nút Leaf (thu thập cảm biến), Relay (chuyển tiếp đa chặng) và Root node (Gateway trung tâm) để truyền dẫn dữ liệu ổn định.

Nhiệm vụ 2: Phát triển Firmware cho Gateway để lập trình cho Root node khả năng điều phối luồng dữ liệu, bóc tách chuỗi JSON từ các nút con và thực hiện giao thức kết nối MQTT.

Nhiệm vụ 3: Tích hợp hệ thống cảnh báo 4G bằng cách cấu hình module A7680C để thực hiện kích bản gọi điện VoLTE và gửi SMS chi tiết thông số đến người dùng ngay khi phát hiện vượt ngưỡng an toàn.

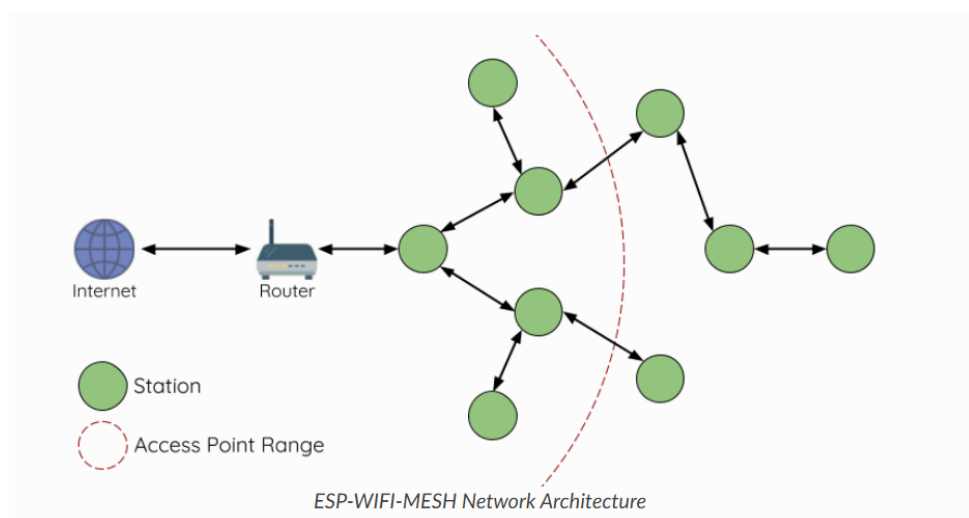
Nhiệm vụ 4: Thiết kế Dashboard bao gồm xây dựng máy chủ Node.js kết hợp cơ sở dữ liệu MongoDB và giao diện Web sử dụng Socket.io để hiển thị biểu đồ thời gian thực.

Nhiệm vụ 5: Thử nghiệm và đánh giá hệ thống của mạng Mesh và đo lường độ trễ của các cảnh báo trong điều kiện thực tế.

2. LÝ THUYẾT

2.1 Giới thiệu tổng quan về ESP MESH

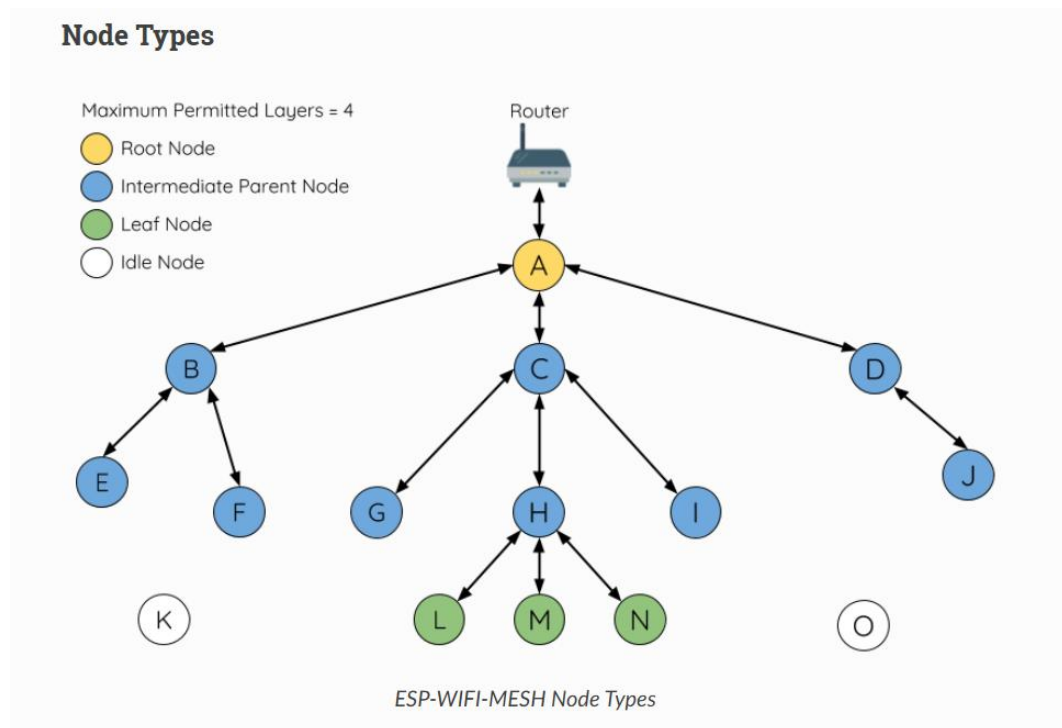
ESP-MESH là một giao thức mạng của Espressif dựa trên Wi-fi. Giao thức này cho phép nhiều thiết bị ESP32 được kết nối với nhau trong một không gian vật lý rộng lớn [1] “được kết nối với nhau thông qua một WLAN (mạng cục bộ không dây) duy nhất”. Chúng làm tăng tầm kết nối của một Router.



Hình 2.1: Mô hình ESP-MESH

ESP-MESH được tổ chức theo mô hình cấu trúc cây (tree topology). Trong cấu trúc này, toàn bộ các thiết bị tham gia mạng mesh được xem như những node (nút mạng) và được phân bổ thành các tầng (layer) khác nhau dựa trên vị trí của chúng trong cây.

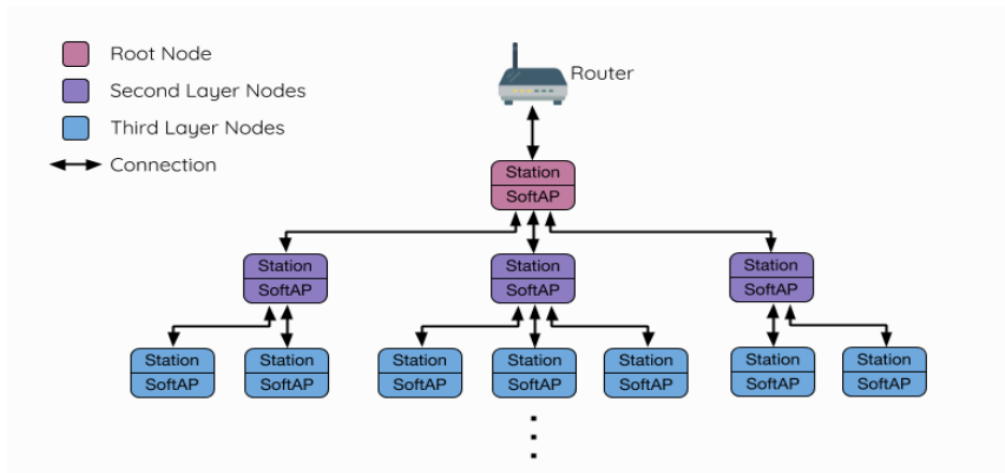
- Root node: Nút duy nhất trong toàn mạng có khả năng kết nối trực tiếp với bộ định tuyến Wi-fi. [2] “chỉ có thể có một nút gốc trong mạng ESP-WIFI_MESH và kết nối ngược dòng của nút gốc chỉ có thể là với bộ định tuyến”.
- Child node: [3] “Bất kỳ nút nào được kết nối với Parent node. Dựa vào Parent node của nó để giao tiếp với phần còn lại của lưới và Root node”
- Parent node: Nút mà cho phép các Child node kết nối vào.
- Leaf node(End node): Nút biên nằm ở tầng dưới cùng của cây. Nút này không nhận thêm Child node và nhiệm vụ chính của nút này là thu thập dữ liệu cảm biến hoặc điều khiển thiết bị sau đó sẽ gửi dữ liệu ngược lên cây thông qua các Parent node.



Hình 2.2: Cấu trúc cây trong ESP-MESH

ESP-WIFI dựa trên cơ chế đặc biệt của ESP32 là có thể hoạt động đồng thời ở chế độ Station(STA) và một AP(Access point). Nhờ vậy, mỗi nút trong mạng MESH đều có hai vai trò cùng lúc:

- Kết nối ngược dòng (upstream): Mỗi nút có duy nhất một liên kết STA để gắn vào Parent node. Đây là đường truyền ngược về phía gốc của mạng.
- Kết nối xuôi dòng (downstream): Mỗi nút mở giao diện softAP của riêng mình để cho phép các Child node kết nối vào. Đây là đường truyền cho các nút cấp dưới.

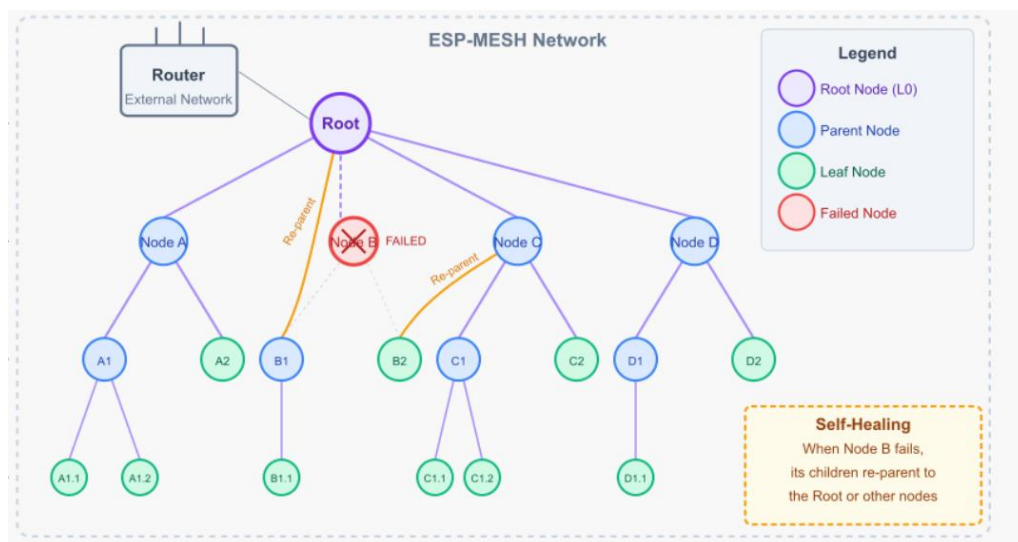


Hình 2.3: ESP-MESH với cơ chế Station-SoftAP đồng thời

ESP-MESH có cơ chế tự tổ chức và cơ chế tự phục hồi. Tự tổ chức là khả năng mạng MESH hình thành cấu trúc cây, gán vai trò cho các nút và duy trì kết nối mà không cần cấu hình thủ công ở tầng ứng dụng. [4] “ ESP-WIFI-MESH là một mạng tự phục hồi, nghĩa là nó có thể phát hiện và sửa chữa lỗi định tuyến. Lỗi xảy ra khi một Parent node có một hoặc nhiều Child node bị hỏng, hoặc khi kết nối giữa Parent node và Child node trở nên không ổn định. Các Child node trong ESP-WIFI-MESH sẽ tự động chọn một Parent node mới và thiết lập kết nối ngược dòng với Parent node đó để duy trì kết nối mạng”.

Cơ chế tự tổ chức và phục hồi của mạng ESP-MESH là một trong những ưu điểm vượt trội giúp hệ thống vận hành ổn định trong môi trường nhà kho phức tạp. Khi một Node mới được khởi tạo, nó không cần sự can thiệp thủ công từ quản trị viên mà sẽ tự động thực hiện quy trình quét các kênh Wi-Fi khả dụng để tìm kiếm gói tin Beacon từ các Parent node tiềm năng. Quy trình này bao gồm việc so sánh cường độ tín hiệu RSSI kết hợp với vị trí tầng (Layer) của Node đó trong cấu trúc cây. Một Node sẽ ưu tiên kết nối với Parent node có Layer thấp hơn (gần

Root hơn) để tối ưu hóa số chặng truyền, từ đó giảm thiểu độ trễ dữ liệu và rủi ro mất gói tin khi truyền tải qua quá nhiều trung gian. Trong trường hợp một Relay node bị ngắt kết nối đột ngột, các Node con thuộc nhánh đó sẽ kích hoạt chế độ tự phục hồi (Self-healing), lập tức tìm kiếm một 'cha' mới trong vùng phủ sóng để tái thiết lập đường truyền về Gateway, đảm bảo tính liên tục của dữ liệu giám sát.



Hình 2.4: Cơ chế tự phục hồi của ESP-MESH

Cơ chế mà Child node chọn Parent node dựa vào các tiêu chí sau:

- RSSI: ưu tiên có cường độ mạnh, liên kết ổn định.

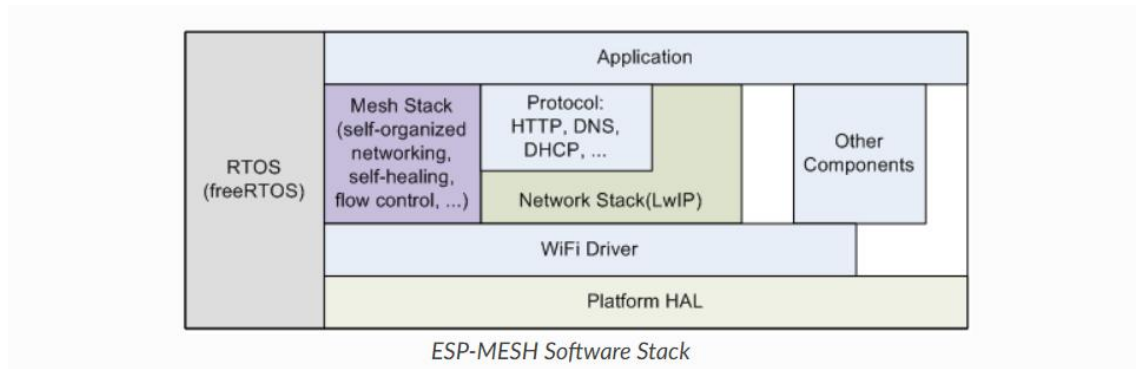
[5] “RSSI (Received Signal Strength Indicator) bản chất của nó là chỉ số cường độ tín hiệu thu hay là chỉ số để độ mạnh của tín hiệu thiết bị thu”.

Bảng 2.1: Đánh giá chất lượng kết nối theo RSSI

Cường độ tín hiệu (RSSI)	Chất lượng	Tốc độ DOWNLOAD	Dịch vụ có thể sử dụng
Lớn hơn -60 dBm	Tốt	80%	Tất cả các dịch vụ: Voice, video, web, mail,..
-61dBm -> -70dBm	Bình thường	60%	Tất cả các dịch vụ: Voice, video, web, mail,...
-71dBm -> -80dBm	Yếu	40%	Web, mail
Nhỏ hơn -80dBm	Rất yếu	N/A	Chạy không ổn định hoặc không hoạt động

- Layer: ưu tiên Parent node có layer nhỏ tức đường truyền đến Root node ngắn hơn, độ trễ thấp hơn.
- Dung lượng: ưu tiên các Parent node có ít Child node đang kết nối sẵn.

[6]“ESP-MESH software stack được xây dựa trên trình điều khiển Wifi/FreeRTOS và có thể sử dụng LwIP stack trong một số trường hợp (ví dụ: Root node)”.



Hình 2.5: ESP-MESH software stack

Stack phần mềm của ESP-WIFI-MESH được xây dựng trên nền tảng trình điều khiển Wi-Fi của ESP32 và hệ điều hành FreeRTOS. Khác với mạng Wi-Fi truyền thống, ESP-MESH không yêu cầu một điểm truy cập trung tâm (AP) để quản lý toàn bộ các trạm (STA). Thay vào đó, nó sử dụng giao thức định tuyến dựa trên cường độ tín hiệu (RSSI) và vị trí tầng (Layer) để xác định đường đi tối ưu.

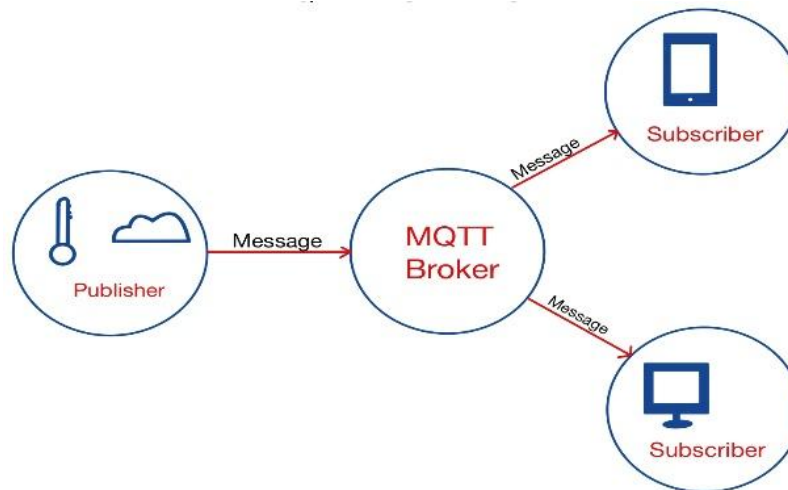
2.2 Giới thiệu về giao thức MQTT

Trong kiến trúc truyền thông của hệ thống IoT, việc lựa chọn một phương thức trao đổi dữ liệu tối ưu giữa các thiết bị đầu cuối và máy chủ là yếu tố tiên quyết nhằm đảm bảo tính ổn định và tiết kiệm tài nguyên. Theo tài liệu tham khảo [7] “MQTT là một giao thức nhắn tin tiêu chuẩn của OASIS dành cho IoT. Giao thức này được thiết kế cực kỳ nhẹ, theo mô hình publish/subscribe rất lý tưởng để kết nối các thiết bị từ xa với mã nguồn nhỏ gọn và băng thông mạng tối thiểu”. Nhờ vào cơ chế trung gian thông qua các chủ đề (Topics), MQTT cho phép tách biệt hoàn toàn thực thể gửi và nhận dữ liệu, giúp giảm thiểu sự phụ thuộc về mặt thời gian và không gian giữa các nút cảm biến có cấu hình phần cứng hạn chế. Để triển

khai thực tế mô hình này, đồ án sử dụng MQTT broker là mosquitto [8] “hỗ trợ giao thức MQTT (v5.0, v3.1.1, v3.1)”. Đây là một giải pháp mã nguồn mở mạnh mẽ, không chỉ đảm bảo khả năng tương thích linh hoạt với nhiều phiên bản giao thức khác nhau mà còn cung cấp hiệu suất xử lý luồng tin nhắn cao, phù hợp để vận hành trong các môi trường mạng có độ trễ lớn hoặc băng thông không ổn định, từ đó tối ưu hóa quá trình giám sát và điều khiển thiết bị từ xa một cách hiệu quả.

Để đảm bảo tính tin cậy của dữ liệu tùy theo yêu cầu của ứng dụng, MQTT cung cấp ba mức QoS (Quality of Service):

- QoS 0 (At most once): Tin nhắn được gửi tối đa một lần, không có phản hồi xác nhận.
- QoS 1 (At least once): Đảm bảo tin nhắn đến đích ít nhất một lần, có cơ chế xác nhận lại.
- QoS 2 (Exactly once): Đảm bảo tin nhắn đến đích chính xác một lần duy nhất.



Hình 2.6: Giao thức MQTT

2.3 Tìm hiểu về các tập lệnh AT

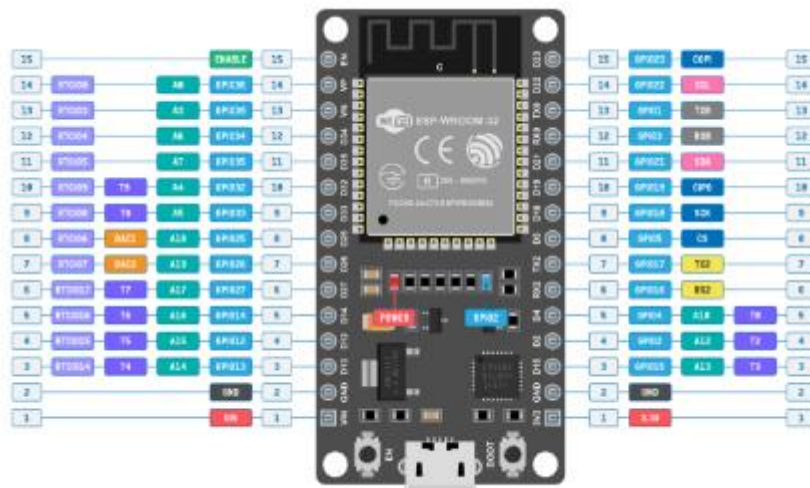
Tập lệnh AT (Attention Commands) là ngôn ngữ giao tiếp chính để vi điều khiển ra lệnh cho module SIM thực hiện các nhiệm vụ cụ thể. Dưới đây là danh sách các lệnh AT quan trọng đã được thực hiện trong firmware của Root node [9]:

- ATD[số điện thoại] : lệnh thực hiện cuộc gọi.
- ATH: lệnh ngắt cuộc gọi.
- AT+CMGF=1: lệnh thiết lập chế độ văn bản cho các tác vụ xử lý tin nhắn SMS.
- AT+CMGS: lệnh khởi tạo quy trình gửi tin nhắn văn bản từ module SIM đến một số điện thoại cụ thể.
- AT+VOLTE=1: lệnh cho module A7680C bật chế độ gọi điện thông qua mạng dữ liệu 4G.
- AT+CTZU=1: Lệnh cho phép module SIM tự động cập nhật thời gian thực và múi giờ từ trạm phát sóng của nhà mạng di động (thông qua giao thức NITZ) ngay khi kết nối vào mạng 4G LTE.

2.4 Tìm hiểu về các linh kiện trong hệ thống

2.4.1 ESP32 Devkit V1

ESP32 Devkit V1 là module phát triển dựa trên vi xử lý ESP32 của Expressif system. Nó là một trong những module phổ biến nhất được sử dụng trong các ứng dụng IoT và nhúng. ESP32 có ưu điểm là cách sử dụng dễ dàng, ra chân đầy đủ và tích hợp mạch nạp vào giao tiếp UART.



Hình 2.7: Sơ đồ chân của ESP32 Devkit V1

- Các thông số kỹ thuật:
 - + CPU: Xtensa Dual-Core LX6 microprocessor 32 bit

- + Tốc độ xung nhịp: 80Mhz – 240Mhz
- + Hỗ trợ 2 giao tiếp không dây: Wifi và Bluetooth
- + Hỗ trợ các loại giao tiếp như UART, SPI, I2C
- + Bảo mật: IEEE 802.11 WFA, WPA/WPA2/WPA3 và WAPI
- + Nhiệt độ hoạt động: -40 - 85 độ C
- + Số cổng GPIO: 30 port
- + Điện áp hoạt động: 2.2 – 3.6V

2.4.2 Cảm biến nhiệt độ và độ ẩm DHT11

Cảm biến DHT11 là một cảm biến nhiệt độ và độ ẩm phổ biến, có khả năng đo nhiệt độ từ 0 – 50 độ C và độ ẩm từ 20% đến 90%.



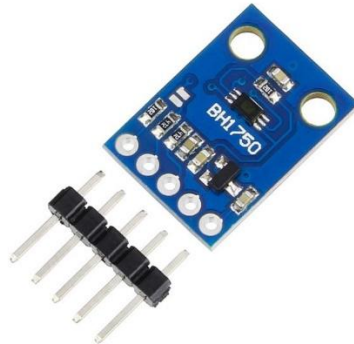
Hình 2.8: Cảm biến DHT11

- Các thông số kỹ thuật:
 - + Dải đo nhiệt độ: 0 – 50 độ C
 - + Dải đo độ ẩm: 20% - 90% RH
 - + Độ chính xác: Nhiệt độ ± 1 độ C và Độ ẩm $\pm 5\%$ RH
 - + Điện áp hoạt động: 3V-5V
 - + Dòng điện tiêu thụ: 2.5mA

2.4.3 Cảm biến ánh sáng BH1750 GY-302

Cảm biến ánh sáng BH1750 GY-302 Digital Light Sensor được sử dụng để đo cường độ ánh sáng theo đơn vị lux, cảm biến có ADC nội và bộ tiền xử lý nên giá

trị được trả ra là giá trị trực tiếp cường độ ánh sáng lux mà không phải qua bất kỳ xử lý hay tính toán nào thông qua giao tiếp I2C.



Hình 2.9: Cảm biến BH1750 GY-302

- Các thông số kỹ thuật:
 - + Điện áp hoạt động: 3 – 5V
 - + Dòng điện: 10mA
 - + Công suất: 50mW
 - + Chuẩn truyền: I2C
 - + Phạm vi hoạt động: 360 độ
 - + Cường độ sáng: 0-65536 Lux

2.4.4 Cảm biến phát hiện chuyển động PIR

Cảm biến thân nhiệt chuyển động PIR (Passive infrared sensor) được sử dụng để phát hiện chuyển động của các vật thể phát ra bức xạ hồng ngoại (con người, con vật, các vật phát nhiệt,...), cảm biến có thể chỉnh được độ nhạy để giới hạn khoảng cách bắt xa gần cũng như cường độ bức xạ của vật thể mong muốn, ngoài ra cảm biến còn có thể điều chỉnh thời gian kích trở (giữ tín hiệu bao lâu sau khi kích hoạt) qua biến trở tích hợp sẵn.



Hình 2.10: Cảm biến PIR

- Các thông số kỹ thuật:
 - + Phạm vi phát hiện: góc 360 độ hình nón, độ xa tối đa 6m
 - + Nhiệt độ hoạt động: 32-122 độ F
 - + Điện áp hoạt động: DC 3.8V – 5V
 - + Mức tiêu thụ dòng: $\leq 50 \mu A$
 - + Thời gian báo: 30 giây có thể tùy chỉnh bằng biến trở
 - + Độ nhạy có thể điều chỉnh bằng biến trở

2.4.5 Module 4G LTE A7680C

Module A7680C là một giải pháp đột phá từ hãng SIMCom, được thiết kế đặc biệt để thay thế các dòng module 2G (như SIM800C) vốn đang dần bị khai tử tại nhiều quốc gia, trong đó có Việt Nam. Module A7680C có khả năng nhắn tin SMS, nghe, gọi, GPRS, ... như một điện thoại nhưng có kích thước nhỏ nhất trong các loại module SIM.



Hình 2.11: Module A7680C

- Các thông số kỹ thuật:
 - + Nguồn cấp: 3.3-4V
 - + Băng tần: LTE-CAT 1 10Mbps, LTE-TDD: B34/B38/B39/B40/B41, LTE-FDD: B1/B3/B5/B8.

2.4.6 Module màn hình OLED V1 0.96 inch

Màn hình Oled 0.96 inch giao tiếp I2C cho khả năng hiển thị đẹp, rõ nét vào ban ngày và khả năng tiết kiệm năng lượng tối đa với mức chi phí phù hợp, màn hình sử dụng giao tiếp I2C cho chất lượng đường truyền ổn định và rất dễ giao tiếp chỉ với 2 chân GPIO.

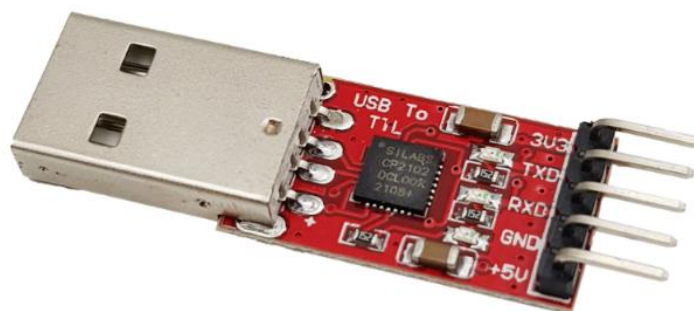


Hình 2.12: Màn hình OLED 0.96 inch

- Các thông số kỹ thuật:
 - + Điện áp hoạt động: 2.2 – 5.5 VDC
 - + Số điểm hiển thị: 128x64 điểm
 - + Độ rộng màn hình: 0.96 inch
 - + Màu hiển thị: Xanh Dương
 - + Driver: SSD1306

2.4.7 Module chuyển đổi USB qua TTL CP2102

Mạch chuyển USB to TTL CP2102 là module chuyển đổi tín hiệu USB sang tín hiệu Serial tuần tự theo chuẩn TTL. Mạch sử dụng chip CP2102. CP2102 là chip USB to UART của Silabs. CP2102 có kích thước nhỏ gọn và yêu cầu rất ít thành phần bên ngoài để hoạt động được ngay.



Hình 2.13: Module chuyển đổi USB qua TTL CP2102

- Các thông số kỹ thuật:
 - + Điện áp hoạt động: 3.3V hoặc 5V
 - + Hỗ trợ hệ điều hành: Windows XP/7/8/10/11, Linux, MacOS
 - + Tốc độ baud: 300bps ~ 1Mbps

2.5 Tổng quan về Website

2.5.1 Giới thiệu tổng quan về Website

Một website được xem là một tập hợp các trang web được liệt kê với nhau và được lưu trữ trên máy chủ web, hỗ trợ truy cập thông qua mạng Internet. Website đóng vai trò quan trọng trong việc là chia sẻ thông tin, cung cấp các dịch vụ và tương tác với người dùng trên thế giới.

Một website thường gồm các thành phần sau:

- + Website: đây là thành phần cơ bản của một website, chứa các nội dung văn bản, hình ảnh, video, âm thanh và các thành phần khác.
- + Địa chỉ website (URL): Mỗi website đều có một địa chỉ duy nhất để truy cập được gọi là URL (Uniform Resource Locator). URL cung cấp định danh cho mỗi website trên Internet.
- + Thiết kế giao diện: Là quyết định về cấu trúc, bố cục, màu sắc và hình ảnh của website. Nó giúp tạo ra trải nghiệm dễ dàng cho người dùng.
- + Nội dung: Nội dung của một website bao gồm thông tin, bài viết, hình ảnh, video, sản phẩm, dịch vụ hoặc bất kỳ loại thông tin nào muốn chia sẻ đến người dùng.
- + Chức năng và tương tác: Website có thể cung cấp các chức năng tương tác như biểu mẫu liên hệ, tìm kiếm, đăng ký thành viên, mua hàng trực tuyến và nhiều tính năng khác. Điều này cho phép người dùng tương tác và thực hiện hoạt động trên website.

Một website hoạt động trên World Wide Web (WWW), bao gồm ba thành phần chính:

- + Tên miền: Địa chỉ duy nhất của website trên Internet.
- + Dịch vụ lưu trữ: máy chủ website lưu trữ các tệp tin và dữ liệu của website để có thể truy cập từ mạng Internet.
- + Mã nguồn: Bao gồm các tệp tin HTML, CSS, Javascript và các ngôn ngữ lập trình khác, định nghĩa cấu trúc và hiển thị website.

2.5.2 Ngôn ngữ HTML

HTML (Hypertext Markup Language) là một ngôn ngữ đánh dấu siêu văn bản. HTML thường được sử dụng để tạo và cấu trúc các phần tử trong website và ứng dụng, phân chia các đoạn văn, tiêu đề, liên kết, trích dẫn các khối và những các khác nữa.

HTML không phải là một ngôn ngữ lập trình mà chỉ là một ngôn ngữ đánh dấu. Có nghĩa là HTML không thể thực hiện các chức năng động mà một ngôn ngữ lập trình có thể làm được. Thay vì thế, HTML tương tự như phần mềm Microsoft Word, chỉ có tác dụng định dạng các thành phần đó trong website.

HTML có ba mục đích chính sau đây:

- + Tạo nội dung: HTML cho phép tạo ra các phần tử nội dung như đoạn văn bản, hình ảnh, video, bảng, danh sách và biểu đồ. Các thẻ HTML được sử dụng để đánh dấu và cấu trúc các thành phần này trong website.
- + Thiết kế giao diện: Mặc dù HTML không phải là ngôn ngữ thiết kế giao diện chuyên sâu, nó cung cấp các phần tử, thuộc tính để định dạng và bố cục website. Sử dụng CSS một ngôn ngữ đồng cấu bạn có thể áp dụng các kiểu và hiệu ứng để tạo ra giao diện hấp dẫn cho website.
- + Lập trình tương tác cho website: Mặc dù HTML không thể thực hiện các chức năng lập trình phức tạp, nó cung cấp các phần tử và thuộc tính cho việc tạo ra các liên kết, biểu mẫu, nút và các phần tử tương tác khác trên website. Để thực

hiện các chức năng tương tác phức tạp hơn, người phát triển thường sử dụng Javascript, một ngôn ngữ lập trình phía máy khách.

2.5.3 Ngôn ngữ CSS

CSS (Cascading Style Sheets) là một ngôn ngữ lập trình được sử dụng để định dạng và trình bày các phần tử trên website. CSS điều khiển cách mà phần tử HTML hiển thị trên trình duyệt, cho phép bạn thay đổi màu sắc, kích thước, phông chữ, khoảng cách và nhiều thuộc tính khác của các phần tử website.

CSS có nhiều ưu điểm và tiện ích. Nó có thể kiểm soát giao diện của tất cả các trang trên một website. Các tập tin CSS được lưu trữ dưới dạng các tệp tin có phần mở rộng .css và có thể được liên kết và áp dụng cho các trang HTML.

Với CSS, bạn không cần phải khai báo thuộc tính cho từng thẻ HTML mỗi khi sử dụng. Thay vào đó, bạn chỉ cần viết các thuộc tính của thẻ trong CSS và nó sẽ được áp dụng tự động mỗi khi thẻ xuất hiện trên website. Điều này giúp giảm số lượng mã code cần viết và tăng tốc độ tải của website.

Bên cạnh đó, CSS cũng hỗ trợ các khái niệm như kế thừa và ưu tiên, cho phép bạn áp dụng các quy tắc chung cho một nhóm phần tử và ghi đè lên các quy tắc cụ thể cho các phần tử riêng biệt.

2.5.4 Ngôn ngữ Javascript

JavaScript (JS) là một ngôn ngữ lập trình kịch bản được sử dụng phổ biến để tạo ra các hiệu ứng tương tác và xử lý logic trên website. Nếu HTML xây dựng cấu trúc và CSS định dạng giao diện, thì JavaScript điều khiển các hành vi động, cho phép website phản hồi lại các thao tác của người dùng như nhấp chuột, nhập dữ liệu hoặc tải nội dung mới mà không cần tải lại trang.

JavaScript sở hữu nhiều ưu điểm vượt trội nhờ khả năng chạy trực tiếp trên trình duyệt của người dùng. Các mã nguồn JavaScript thường được lưu trữ trong các tệp tin có phần mở rộng .js và có thể được tích hợp dễ dàng vào các trang HTML. Điều này giúp giảm tải cho máy chủ và mang lại trải nghiệm mượt mà, tức thì cho người sử dụng.

Với JavaScript, bạn có thể biến một trang web tĩnh thành một ứng dụng web mạnh mẽ. Thay vì chỉ hiển thị thông tin, bạn có thể xây dựng các tính năng phức tạp như thanh cuộn hình ảnh, bản đồ tương tác, các biểu mẫu kiểm tra dữ liệu tự động hoặc thậm chí là phát triển trò chơi trên trình duyệt. Nhờ hệ sinh thái thư viện và framework đồ sộ (như React, Vue hay Angular), việc phát triển các tính năng hiện đại trở nên nhanh chóng và tối ưu hơn.

Bên cạnh đó, JavaScript là một ngôn ngữ linh hoạt, hỗ trợ nhiều phong cách lập trình khác nhau từ hướng đối tượng đến hướng hàm. Khả năng xử lý bất đồng bộ (asynchronous) của nó cho phép website thực hiện nhiều tác vụ cùng lúc, giúp tăng hiệu suất và đảm bảo sự liền mạch trong quá trình tương tác của người dùng.

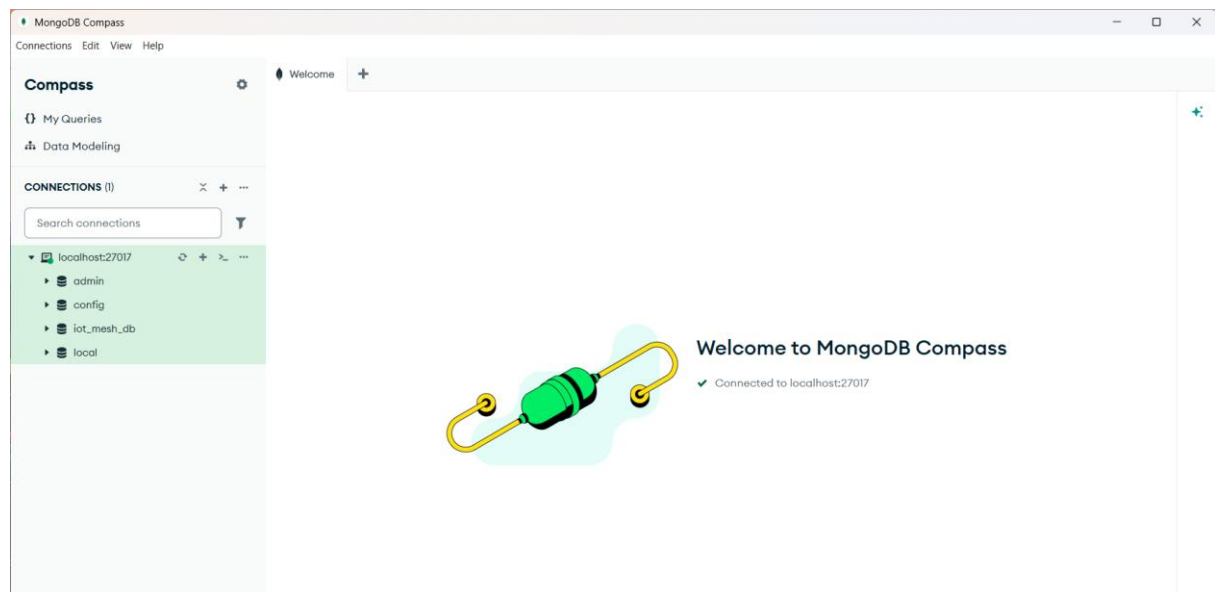
2.5.5 Database MongoDB

MongoDB là hệ quản trị cơ sở dữ liệu NoSQL mã nguồn mở hoạt động theo mô hình hướng tài liệu. Thay vì dùng bảng và hàng như truyền thống, nó lưu trữ dữ liệu dưới dạng các tài liệu linh hoạt tương tự JSON. Điều này cho phép bạn lưu trữ các cấu trúc dữ liệu đa dạng mà không cần khai báo lược đồ cứng nhắc.

Điểm mạnh của MongoDB nằm ở hiệu suất cao và khả năng mở rộng linh hoạt. Dữ liệu được tổ chức thành các tập hợp giúp tối ưu tốc độ truy xuất cho các ứng dụng thời gian thực hoặc xử lý dữ liệu lớn. Bạn có thể dễ dàng nâng cấp hệ thống bằng cách phân tán dữ liệu trên nhiều máy chủ một cách tự nhiên.

Việc phát triển ứng dụng trở nên trực quan hơn nhờ cấu trúc dữ liệu tương thích tốt với các ngôn ngữ lập trình hiện đại. Thay vì các phép nối phức tạp, bạn có thể nhúng dữ liệu liên quan vào một tài liệu duy nhất để tăng tốc độ truy vấn và giảm độ trễ. Điều này giúp lập trình viên linh hoạt hơn trong việc quản lý và thay đổi cấu trúc thông tin.

Bên cạnh đó, MongoDB đảm bảo tính sẵn sàng cao nhờ cơ chế sao lưu tự động và khả năng chịu lỗi tốt. Đây là giải pháp mạnh mẽ cho các dự án từ ứng dụng web quy mô lớn đến các hệ thống quản lý dữ liệu từ cảm biến.



Hình 2.14: Database MongoDB

3. TỔNG QUAN VỀ HỆ THỐNG

- Cấu hình cho hệ thống:

Mục tiêu: Đảm bảo hệ thống chạy theo mô hình Leaf Node -> Relay Node -> Root Node -> Server

Quy tắc kết nối: Bật Root Node trước sau đó cho 2 Relay Node kết nối Root Node và cuối cùng là để Leaf Node tự dò để kết nối đến 1 trong 2 Relay Node. Số hop của hệ thống là 2.

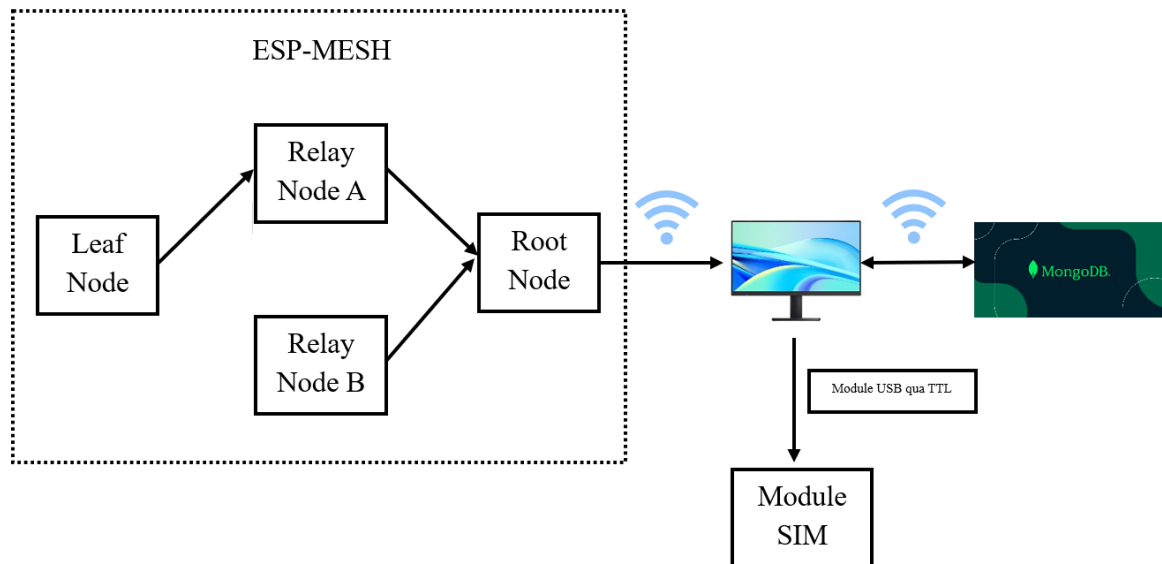
Root Node: Sẽ được cấu hình chỉ kết nối với 2 Child Node để đảm bảo chạy theo mô hình và cấu hình là Root Node duy nhất. Vì nếu cấu hình kết nối được nhiều Child Node hơn thì Leaf Node nếu ở gần sẽ dò và kết nối thẳng đến Root Node và không chạy theo mô hình mong muốn.

Relay Node: Sẽ được cấu hình kết nối 6 Child Node. Theo lý thuyết thì mỗi node có thể kết nối được 10 Child Node nhưng theo nhà sản xuất để hệ thống chạy ổn định chỉ nên để mỗi Node có thể kết nối được 4-6 Child Node.

Leaf Node: Sẽ được cấu hình mặc định với vai trò Leaf Node chỉ thu thập dữ liệu môi trường và không kết nối bất kỳ Child Node nào.

Việc cấu hình vai trò mặc định cho các Node để tránh việc Node dò không tìm thấy Parent Node và thực hiện chức năng tự tổ chức để trở thành 1 nhánh mạng riêng.

- Quá trình hệ thống hoạt động:



Hình 3.1: Sơ đồ tổng quát hệ thống

Root Node: Khởi động sẽ kết nối Wifi từ AP phát ra và cho phép các Child Node kết nối vào. Sau khi mạng Mesh đã hình thành thì nó sẽ nhận dữ liệu từ các Child Node gửi lên và đẩy dữ liệu đó lên Server xử lý.

Relay Node: Gồm Relay Node A và Relay Node B sẽ kết nối vào Root Node và sau khi kết nối nó bắt đầu phát tín hiệu để kết nối các Child Node. Khi nhận được dữ liệu từ Leaf Node nó sẽ chuyển tiếp dữ liệu đó lên Root Node.

Leaf Node: Vì 2 Relay ở cùng tầng với nhau vậy khoảng cách gần nhất đến Root Node là như nhau và 2 Relay Node mới chỉ kết nối đến Root Node. Do đó khi Leaf Node dò các Parent Node xung quang nó sẽ dựa vào RSSI tại thời điểm nó dò để so sánh xem Relay Node nào mạnh hơn mà kết nối. Sau khi kết nối được với Relay Node mạng Mesh đã hình thành nó sẽ thực hiện đọc giá trị cảm biến và gửi lên Root Node.

Server: Nhận dữ liệu từ Root Node thông qua giao thức MQTT và xử lý các dữ liệu đó nếu phát hiện vượt ngưỡng an toàn thì sẽ dùng Module SIM cảnh báo đến người dùng. Cũng có thể chỉnh sửa ngưỡng an toàn ngay tại website của Server.

Database: Lưu trữ các dữ liệu do mạng Mesh gửi lên và lưu các thông tin của khách hàng để thực hiện cảnh báo đến từng người dùng khác nhau.

4. THIẾT KẾ VÀ THỰC HIỆN PHẦN CỨNG

4.1 Yêu cầu thiết kế

Hệ thống có Leaf Node là node thu thập dữ liệu cảm biến từ môi trường như nhiệt độ, độ ẩm, ánh sáng và chuyển động. 2 Relay Node để chuyển tiếp dữ liệu từ Leaf Node đến Root Node, 2 Relay Node nhằm mục đích để Leaf Node so sánh RSSI của bên nào mạnh hơn mà kết nối.

Hệ thống có 1 Gateway là Root Node tổng hợp dữ liệu từ các node gửi lên server để lưu trữ, giám sát trên website và phát cảnh báo khi vượt ngưỡng cho phép.

Hệ thống có màn hình hiển thị tại Leaf Node để người kiểm tra tại hiện trường có thể xem được mà không cần dùng đến website.

4.2 Lựa chọn linh kiện và phân tích

Việc lựa chọn vi điều khiển phù hợp là bước quyết định nhằm đảm bảo khả năng xử lý dữ liệu và tính ổn định của toàn hệ thống.

Các phương án lựa chọn:

Bảng 4.1: Phương án chọn vi điều khiển cho hệ thống

Board	Raspberry Pi 4 Model B	ESP32	ESP8266
-------	------------------------	-------	---------

Phần cứng	Ngoại vi đa dạng, khả năng xử lý mạnh mẽ, có chức năng kết nối Wifi	Ngoại vi ít, khả năng xử lý trung bình, có chức năng kết nối Wifi	Ngoại vi ít, khả năng xử lý thấp, có chức năng kết nối Wifi
Phần mềm	Thư viện hỗ trợ không nhiều, lập trình phức tạp	Thư viện hỗ trợ nhiều, nhiều loại IDE phát triển	Thư viện hỗ trợ nhiều, nhiều loại IDE phát triển
Giá thành	Giá thành rất cao	Giá thành thấp	Giá thành thấp

Dựa vào bảng trên, em quyết định chọn ESP32 làm vi điều khiển cho hệ thống với ưu điểm giá rẻ, chức năng phù hợp với yêu cầu của hệ thống.

Sở dĩ ESP32 được lựa chọn thay vì các dòng vi điều khiển 8-bit hay ESP8266 là nhờ kiến trúc Dual-core mạnh mẽ. Việc chạy giao thức ESP-MESH đòi hỏi tài nguyên bộ nhớ và khả năng xử lý ngắt liên tục; với ESP32, nhân lõi thứ hai có thể được dành riêng cho các tác vụ lấy mẫu cảm biến và điều khiển module SIM thông qua lệnh AT, giúp tránh tình trạng xung đột tài nguyên. Bên cạnh đó, việc chọn module A7680C thay cho các dòng SIM 2G cũ là một bước đi đón đầu công nghệ. Trong bối cảnh các nhà mạng đang dần cắt sóng 2G, việc sử dụng LTE Cat 1 không chỉ đảm bảo tốc độ gửi tin nhắn và thực hiện cuộc gọi nhanh chóng (trong khoảng 10 giây) mà còn giúp hệ thống có vòng đời sử dụng dài hơn trong thực tế.

Việc lựa chọn cảm biến nhiệt độ phù hợp là yếu tố quan trọng nhằm đảm bảo tính chính xác và kịp thời trong việc giám sát điều kiện vi khí hậu của nhà kho.

Các phương án lựa chọn:

Bảng 4.2: Phương án chọn cảm biến nhiệt độ cho hệ thống

Module	DHT11	DHT22	SHT30
Điện áp hoạt động	3.3V-5V	3.3V-6V	2.4V-5.5V

Dải đo nhiệt độ	0-50 độ C	-40-80 độ C	-40-125 độ C
Dải đo độ ẩm	20-90%	0-100%	0-100%
Độ chính xác	± 2 độ C/ $\pm 5\%$	± 0.5 độ C/ $\pm 2\%$	± 0.3 độ C/ $\pm 3\%$
Giao thức kết nối	Single-bus	Single-bus	I2C
Giá thành	Thấp	Trung bình	Cao

Qua bảng so sánh trên, cảm biến DHT11 được lựa chọn làm linh kiện thu thập dữ liệu chính cho các Leaf Node trong hệ thống. Mặc dù DHT22 và SHT30 có độ chính xác cao hơn, nhưng trong môi trường nhà kho thông thường, các thông số nhiệt độ và độ ẩm thường không biến thiên quá khắc nghiệt vượt ngoài dải đo 0 - 50°C. Với mục tiêu thiết kế một hệ thống giám sát đa điểm với số lượng lớn các nút mạng, việc sử dụng DHT11 giúp tiết kiệm chi phí đáng kể mà vẫn đảm bảo sai số đo lường trong ngưỡng chấp nhận được. Đồng thời, giao thức Single-bus của DHT11 giúp tiết kiệm chân GPIO cho ESP32, tạo điều kiện tích hợp thêm các module khác.

Việc lựa chọn cảm biến ánh sáng phù hợp giúp hệ thống nhận biết chính xác mức độ điện áp thay đổi theo cường độ rọi, từ đó tối ưu hóa việc kiểm soát trạng thái chiếu sáng.

Các phương án lựa chọn:

Bảng 4.3: Phương án chọn cảm biến ánh sáng cho hệ thống

Module	Quang trở LDR	BH1750 GY-302	TSL2561
Nguyên lý hoạt động	Thay đổi điện trở theo ánh sáng	Chuyển đổi quang – số tích hợp	Chuyển đổi quang – số phổ rộng
Tín hiệu đầu ra	Analog	Digital	Digital

Độ nhạy	Cao với ánh sáng khả kiến	Rất cao, có thể đo đơn vị Lux	Cao, đo được cả hồng ngoại
Độ phức tạp lập trình	Thấp	Trung bình	Cao
Điện áp hoạt động	3.3V-5V	3V-5V	2.7V-3.6V
Khả năng điều chỉnh	Bằng biến trở trên module	Bằng phần mềm	Bằng phần mềm

Hệ thống ưu tiên lựa chọn cảm biến cường độ ánh sáng kỹ thuật số BH1750 GY-302. Lý do cốt lõi là cảm biến này sử dụng phương thức chuyển đổi quang - số tích hợp và truyền thông kỹ thuật số thông qua giao tiếp I2C. Việc này giúp loại bỏ hoàn toàn hiện tượng nhiễu tín hiệu Analog và sai số lớn do đặc tính phi tuyến tính của bộ ADC 12-bit tích hợp trên vi điều khiển ESP32 gây ra. Tín hiệu đầu ra từ BH1750 được chuẩn hóa trực tiếp sang đơn vị Lux với độ nhạy rất cao, cung cấp chính xác giá trị định lượng về độ sáng thực tế trong môi trường nhà kho lên Web Dashboard và màn hình OLED thay vì chỉ đo điện áp tương đối như quang trở LDR. Ngoài ra, với dải điện áp hoạt động linh hoạt (3V - 5V) phù hợp với hạ tầng nguồn của hệ thống và độ phức tạp lập trình ở mức trung bình, BH1750 là giải pháp cân bằng hoàn hảo giữa độ tin cậy của phép đo, khả năng điều chỉnh bằng phần mềm và chi phí đầu tư linh kiện cho một hệ thống giám sát công nghiệp.

Việc lựa chọn cảm biến phát hiện chuyển động phù hợp là bước then chốt giúp nâng cao khả năng tự động hóa và đảm bảo an ninh cho khu vực giám sát.

Các phương án lựa chọn:

Bảng 4.4: Phương án cảm biến phát hiện chuyển động cho hệ thống

Module	PIR HC-SR501	PIR HC-SR505	RCWL-0516
Điện áp hoạt động	3.8V-20V	4.5V-18V	4V-28V

Góc quét	Dưới 120 độ	Dưới 100 độ	360 độ
Khoảng cách phát hiện	Tối đa 7m	Tối đa 3m	5m-9m
Khả năng điều chỉnh	Chỉnh được độ nhạy và thời gian trễ	Không hỗ trợ chỉnh biến trở	Không hỗ trợ
Kích thước	Trung bình	Nhỏ	Nhỏ
Giá thành	Trung bình	Trung bình	Trung bình

Cảm biến HC-SR501 được lựa chọn nhờ ưu điểm vượt trội về khả năng tùy biến thông qua hai biến trở tích hợp trên board mạch. Điều này cho phép người vận hành điều chỉnh độ nhạy để phù hợp với diện tích từng khu vực cụ thể trong nhà kho và thiết lập thời gian delay lên đến 30 giây sau khi phát hiện chuyển động. So với các dòng mini như HC-SR505, HC-SR501 mang lại độ tin cậy cao hơn, hạn chế tối đa các tình trạng cảnh báo sai lệch do nhiễu môi trường, từ đó tối ưu hóa độ chính xác cho hệ thống cảnh báo qua SMS và VoLTE.

Việc lựa chọn thiết bị hiển thị phù hợp đóng vai trò trực quan hóa dữ liệu, giúp người quản lý dễ dàng theo dõi và tương tác với các thông số theo thời gian thực.

Các phương án lựa chọn:

Bảng 4.5: Phương án chọn thiết bị hiển thị cho hệ thống

Module	Màn hình OLED 0.96'' SSD1306	Màn hình LCD 16x2	Màn hình TFT 1.8'' ST7735
Công nghệ hiển thị	OLED	LCD	TFT color
Độ phân giải	128x64 pixel	16 ký tự x 2 dòng	128x160 pixel

Giao thức kết nối	I2C	I2C (module rời)	SPI
Tiêu thụ điện năng	Rất thấp	Trung bình	Cao
Giá thành	Trung bình	Trung bình	Cao

Màn hình OLED 0.96 inch sử dụng driver SSD1306 được chọn vì tính tương thích hoàn hảo với vi điều khiển ESP32 và khả năng tiết kiệm điện năng tối ưu. Công nghệ OLED không cần đèn nền, giúp các thông số như nhiệt độ, độ ẩm và trạng thái "CÓ NGƯỜI" hiển thị rõ nét ngay cả trong môi trường ánh sáng yếu của nhà kho. Đặc biệt, giao thức I2C chỉ sử dụng 2 chân GPIO (SDA và SCL) giúp đơn giản hóa việc thiết kế mạch in PCB và dành tài nguyên cho các linh kiện quan trọng khác như cảm biến và module 4G.

Việc lựa chọn module SIM phù hợp là điều kiện tiên quyết giúp hệ thống duy trì kênh truyền thông độc lập và thực hiện các kịch bản cảnh báo khẩn cấp qua mạng viễn thông.

Các phương án lựa chọn:

Bảng 4.6: Phương án chọn module Sim cho hệ thống

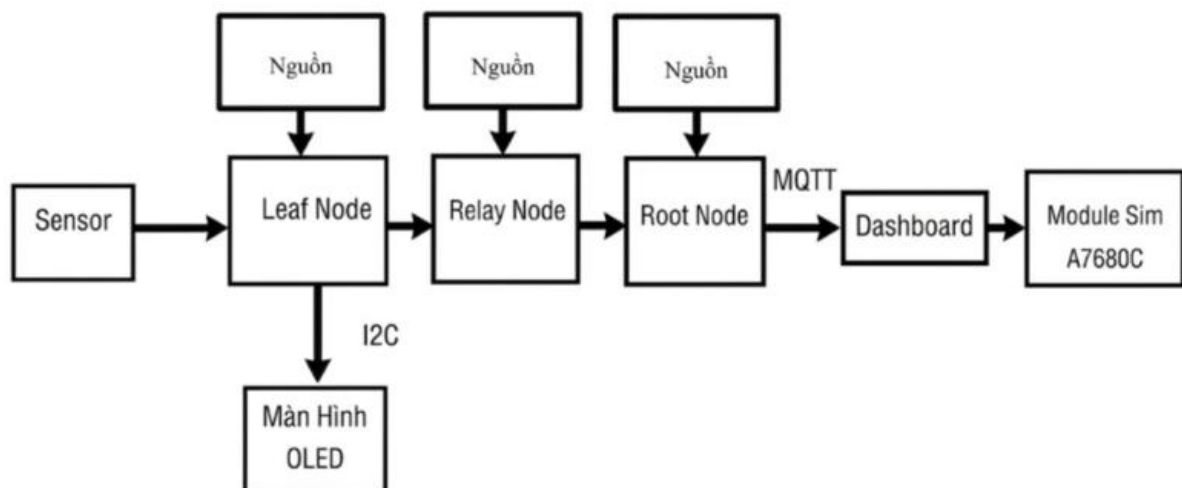
Module	SIM800L	SIM7600CE	A7680C
Công nghệ mạng	2G	4G LTE Cat 4	4G LTE Cat 1
Tốc độ truyền dữ liệu	Rất chậm	Rất nhanh (150Mbps)	Trung bình (10Mbps)
Tiêu thụ năng lượng	Thấp	Rất cao	Trung bình
Kích thước	Nhỏ	Lớn	Siêu nhỏ
Giá thành	Giá thành thấp	Giá thành rất cao	Giá thành không quá cao

Dựa vào bảng trên, em quyết định chọn module Sim A7680C dùng trong hệ thống với ưu điểm về kích thước và giá thành không quá cao, chức năng truyền dữ liệu phù hợp với hệ thống.

Việc lựa chọn A7680C thay vì các dòng SIM800L (2G) truyền thống là một bước đi bắt buộc trong bối cảnh các nhà mạng tại Việt Nam đang dần lộ trình tắt sóng 2G.

- Công nghệ VoLTE: Cho phép thực hiện cuộc gọi cảnh báo trên nền tảng dữ liệu 4G với chất lượng âm thanh cao và thời gian kết nối nhanh hơn.
- Thiết kế tối ưu: A7680C có kích thước siêu nhỏ (15.7x17.6 mm), tương thích chân cắm với một số dòng module 2G cũ, giúp dễ dàng nâng cấp phần cứng.

4.3 Sơ đồ khối tổng quát



Hình 4.1: Block diagram

Leaf Node: Thu thập các giá trị nhiệt độ, độ ẩm, ánh sáng, so sánh xem Relay Node nào có RSSI lớn hơn để kết nối và chuyển động để gửi dữ liệu về cho Root Node.

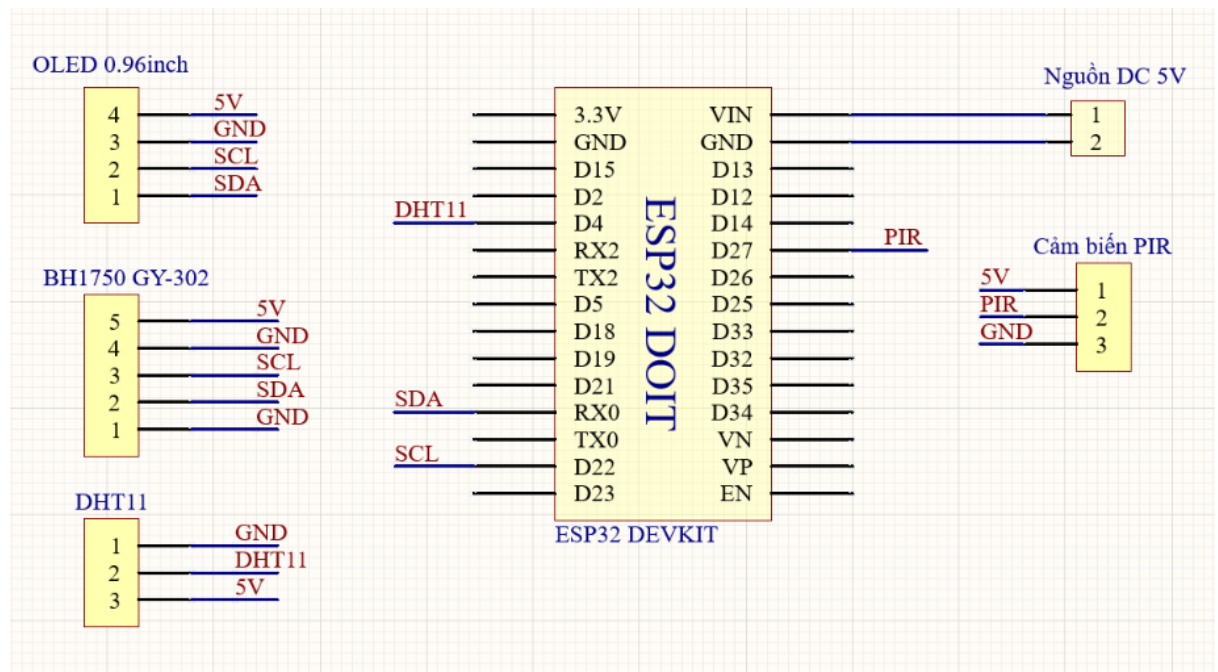
Relay Node: Chuyển tiếp dữ liệu từ Leaf Node đến Root Node.

Root Node: Nhận dữ liệu từ các node gửi lên và gửi dữ liệu lên server.

Dashboard: Hiển thị dữ liệu trên website, có thể thay đổi ngưỡng trực tiếp trên website và cảnh báo đến người dùng khi vượt ngưỡng an toàn.

4.4 Thiết kế mạch chi tiết

4.4.1 Thiết kế mạch cho Leaf Node

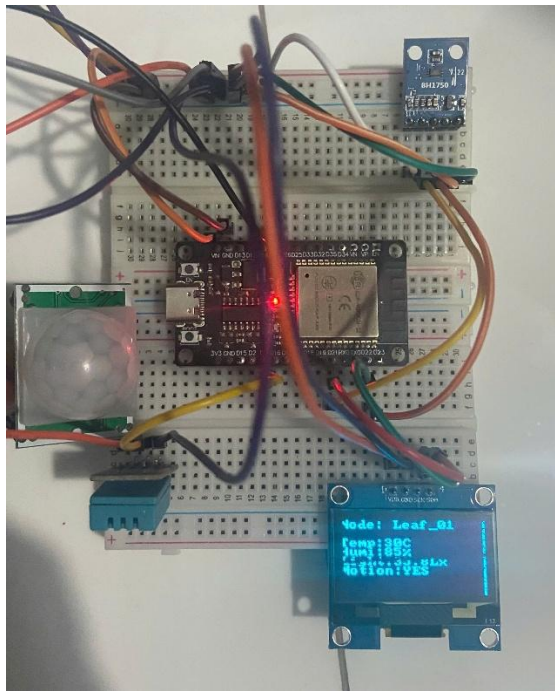


Hình 4.2: Schematic của Leaf Node

Tính toán thông số của mạch:

- Vi điều khiển ESP32
Điện áp cấp: 5V
Dòng tiêu thụ tối thiểu lúc truyền: 160mA
- Màn hình OLED
Điện áp cấp: 5V
Dòng tiêu thụ tối thiểu: 10mA
- Cảm biến nhiệt độ DHT11
Điện áp cấp: 5V
Dòng tiêu thụ tối thiểu: 2.5mA
- Cảm biến ánh sáng BH1750 GY-302
Điện áp cấp: 5V
Dòng tiêu thụ tối thiểu: 10mA
- Cảm biến phát hiện chuyển động PIR
Điện áp cấp: 5V

Dòng tiêu thụ tối thiểu: 65mA



Hình 4.3: Leaf Node thực tế

4.4.2 Thiết kế mạch cho Relay Node

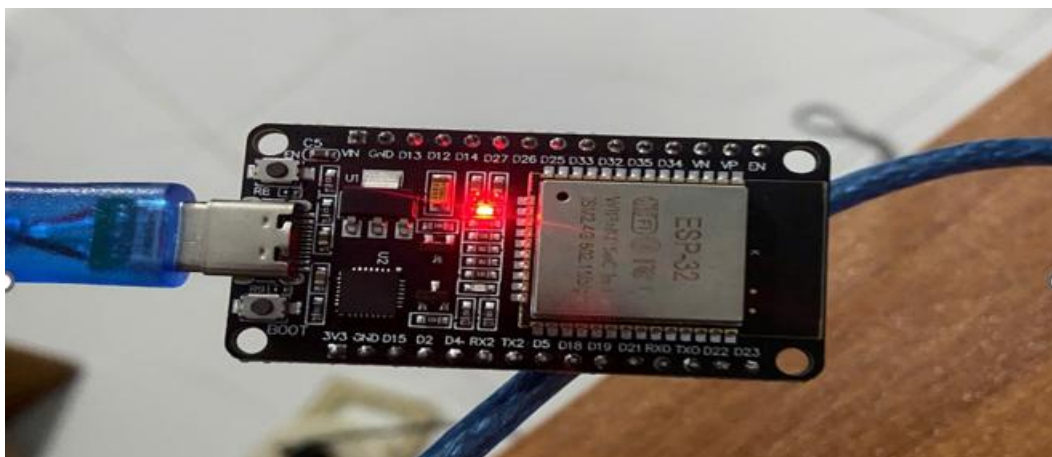
Tính toán thông số mạch:

- Vi điều khiển ESP32

Điện áp cấp: 5V

Dòng tiêu thụ tối thiểu lúc truyền: 160mA

Dòng tiêu thụ tối thiểu lúc nhận: 95mA



Hình 4.4: Relay Node thực tế

4.4.3 Thiết kế mạch cho Root Node

Tính toán thông số mạch:

- Vi điều khiển ESP32

Điện áp cấp: 5V

Dòng tiêu thụ tối thiểu lúc truyền: 160mA

Dòng tiêu thụ tối thiểu lúc nhận: 95mA



Hình 4.5: Root Node thực tế

4.4.4 Module Sim cảnh báo từ Server

- Module Sim A7680C

Điện áp cấp: 3.3V

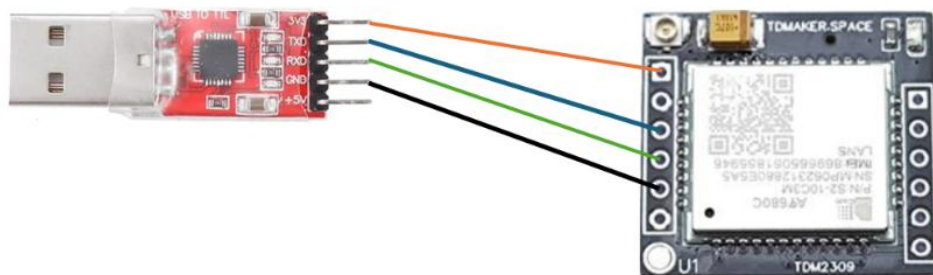
Dòng tiêu thụ tối thiểu lúc truyền: 100mA

Dòng tiêu thụ tối thiểu lúc nhận: 20mA

- Module chuyển đổi USB qua TTL CP2102

Điện áp cấp: 5V

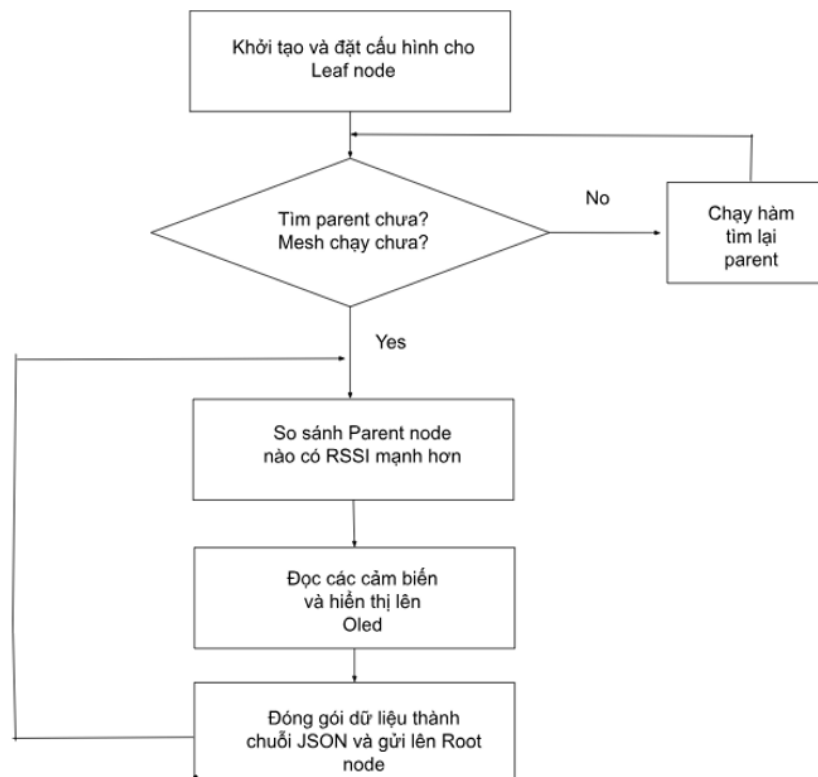
Dòng tiêu thụ tối thiểu: 20mA



Hình 4.6: Module SIM ở server

5. THIẾT KẾ VÀ THỰC HIỆN PHẦN MỀM

5.1 Lưu đồ giải thuật của Leaf Node



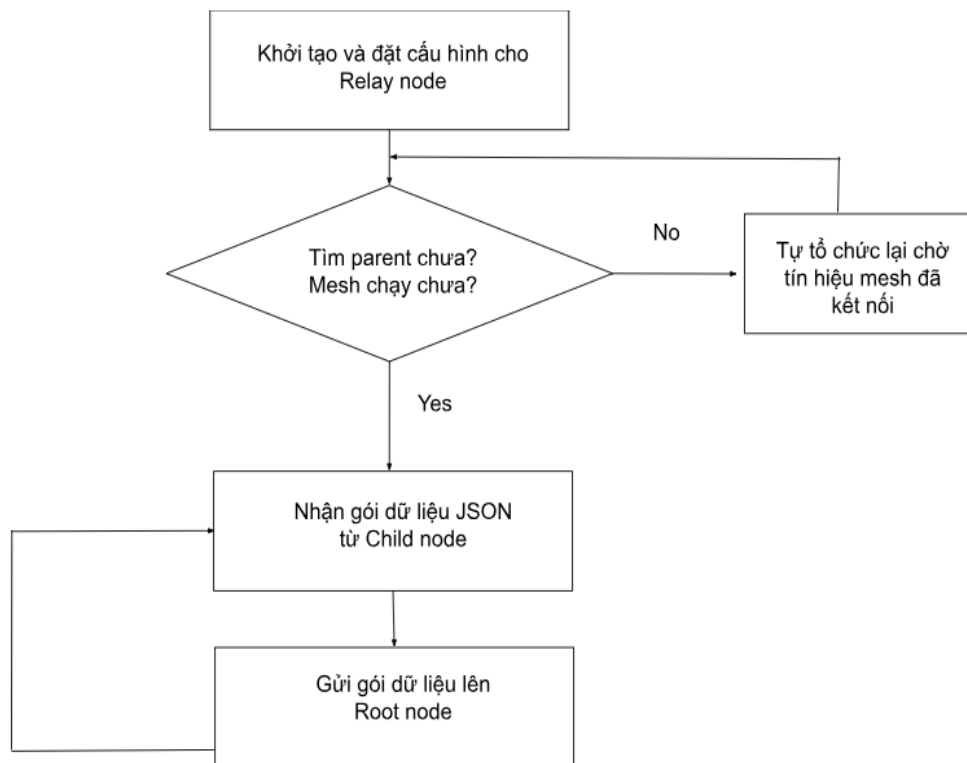
Hình 5.1: Lưu đồ giải thuật Leaf node

Chương trình hoạt động cho Leaf Node theo thứ tự sau:

- Khởi tạo hệ thống: Node thực hiện cấu hình các thông số mạng MESH, thiết lập vai trò thiết bị và khởi động các cảm biến cùng màn hình OLED. Bước này đảm bảo mọi thành phần phần cứng và phần mềm đều sẵn sàng trước khi gia nhập mạng.
- Kiểm tra và kết nối mạng: Node liên tục giám sát trạng thái để xác nhận đã kết nối thành công với parent node hay chưa. Nếu chưa, thiết bị sẽ tự động kích hoạt quy trình quét kênh và thử kết nối lại cho đến khi đường truyền được thiết lập.
- Tối ưu hóa đường truyền: Hệ thống thực hiện so sánh chỉ số RSSI giữa các parent node khả dụng để lựa chọn điểm kết nối có cường độ tín hiệu mạnh nhất. Việc này giúp duy trì sự ổn định của mạng và giảm thiểu rủi ro mất gói tin khi truyền tải xa.

- Xử lý dữ liệu cảm biến: Node tiến hành đọc thông số từ cảm biến, hiển thị giá trị lên màn hình OLED và đóng gói thông tin theo định dạng JSON. Cấu trúc JSON giúp dữ liệu trở nên tường minh, tạo điều kiện thuận lợi cho việc xử lý tại Root node và máy chủ.
- Truyền tin và tự phục hồi: Gói tin được gửi theo phương thức P2P trực tiếp về Root node để cập nhật lên hệ thống giám sát. Toàn bộ chu trình được lặp lại liên tục, cho phép node tự động quay lại bước tìm kiếm nếu xảy ra sự cố mất kết nối đột ngột.

5.2 Lưu đồ giải thuật của Relay Node



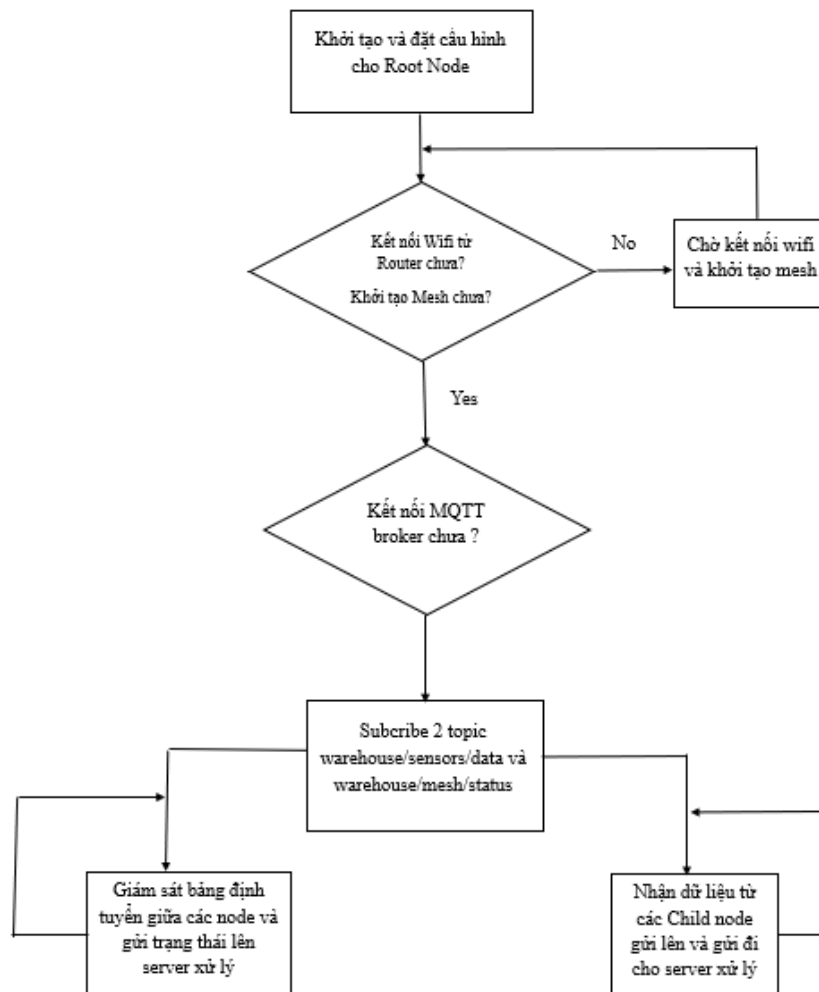
Hình 5.2: Lưu đồ giải thuật Relay node

Chương trình hoạt động cho Relay Node theo thứ tự sau:

- Khởi tạo hệ thống và điểm truy cập: Node thực hiện cấu hình bộ nhớ NVS, Wi-Fi và các tham số mạng Mesh, đồng thời kích hoạt chế độ SoftAP. Việc phát SoftAP cho phép Relay node sẵn sàng đóng vai trò làm trung gian để tiếp nhận các kết nối từ các node con trong cấu trúc mạng.

- Duy trì trạng thái kết nối: Thiết bị liên tục kiểm tra sự hiện diện của parent node và trạng thái hoạt động của hệ thống Mesh. Nếu kết nối chưa được thiết lập, cơ chế tự tổ chức sẽ tự động tìm kiếm và nối lại mạng trước khi quay về bước kiểm tra ban đầu.
- Tiếp nhận dữ liệu từ nhánh con: Sau khi gia nhập mạng ổn định, Relay node chuyển sang trạng thái lắng nghe để nhận các gói tin định dạng JSON từ các Child node gửi đến. Các gói tin này được lưu trữ tạm thời trong bộ đệm để chuẩn bị cho quá trình chuyển tiếp lên các cấp cao hơn.
- Chuyển tiếp dữ liệu: Nếu đã xác định được địa chỉ đích, dữ liệu sẽ được đẩy trực tiếp về Root node để tối ưu hóa tốc độ truyền tải. Trong trường hợp chưa biết địa chỉ Root, node sẽ gửi gói tin cho parent node để hệ thống tự động định tuyến theo cấu trúc mạng.
- Vòng lặp vận hành và tự phục hồi: Toàn bộ quy trình nhận và gửi dữ liệu được thực hiện liên tục để đảm bảo luồng thông tin xuyên suốt. Nếu xảy ra sự cố mất kết nối với parent node, Relay node sẽ tự động quay lại nhánh tự tổ chức để khôi phục liên kết và tiếp tục nhiệm vụ chuyển tiếp.

5.3 Lưu đồ giải thuật của Root Node



Hình 5.3: Lưu đồ giải thuật Root node

Chương trình hoạt động cho Root Node theo thứ tự sau:

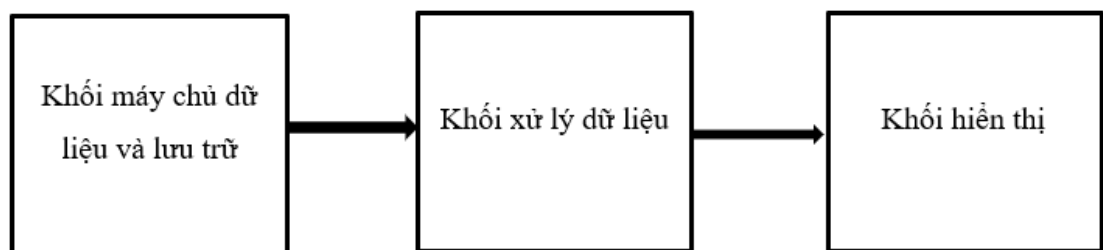
- Khởi tạo hệ thống và kết nối mạng: Root node thực hiện cấu hình Wi-Fi và mạng Mesh, sau đó thiết lập kết nối với Router để lấy địa chỉ IP. Nếu chưa nhận được IP, hệ thống sẽ ở trạng thái chờ sự kiện IP_EVENT_STA_GOT_IP trước khi chuyển sang các bước xử lý tiếp theo.
- Thiết lập giao thức MQTT: Sau khi có địa chỉ IP, node tiến hành khởi tạo và kết nối tới MQTT broker để chuẩn bị cho việc trao đổi dữ liệu với Dashboard. Quá trình này được thực hiện lặp lại liên tục cho đến khi kết nối thành công nhằm đảm bảo tính sẵn sàng của đường truyền.
- Chuyển tiếp dữ liệu thời gian thực: Root Node kích hoạt tác vụ lắng nghe để thu thập các gói tin cảm biến từ các nút con gửi lên qua mạng nội bộ.

Ngay khi nhận được dữ liệu, Root Node đóng vai trò cổng trung gian để chuyển tiếp nguyên văn chuỗi JSON này lên chủ đề quy ước của MQTT Broker nhằm đẩy về cho máy chủ xử lý tính toán.

- **Giám sát và báo cáo sự cố hạ tầng:** Root Node liên tục theo dõi biến động của bảng định tuyến trong mạng Mesh. Khi xảy ra sự cố mất nút trung gian hoặc khi mạng tự phục hồi liên kết thành công, Root Node sẽ lập tức phát thông điệp trạng thái mạng lên MQTT Broker để máy chủ trung tâm nhận diện và kích hoạt các kịch bản cứu hộ khẩn cấp qua module SIM.

- **Duy trì vận hành và tự phục hồi:** Toàn bộ chu trình vận hành được lặp lại liên tục để đảm bảo hệ thống luôn hoạt động ổn định và chính xác. Trong trường hợp bị mất IP hoặc ngắt kết nối MQTT, node sẽ tự động quay lại các bước kiểm tra tương ứng để khôi phục trạng thái kết nối ban đầu.

5.4 Sơ đồ khối của Dashboard



Hình 5.4: Sơ đồ khối của Dashboard

- **Khối máy chủ dữ liệu và lưu trữ:** Đây là trung tâm tiếp nhận, xử lý logic cốt lõi và quản lý thông tin của toàn bộ hệ thống, được xây dựng trên nền tảng Node.js và Express. Khối này sử dụng thư viện MQTT.js để kết nối tới MQTT Broker, đóng vai trò thu thập dữ liệu thông qua hai chủ đề chuyên biệt: `warehouse/sensors/data` (dữ liệu từ các Leaf Node) và `warehouse/mesh/status` (trạng thái cấu trúc mạng và tiến trình Self-Healing từ Root Node). Tại đây, máy chủ đóng vai trò là "bộ não" thực hiện thuật toán kiểm tra ngưỡng an toàn bằng cách tra cứu động cấu hình riêng biệt của từng `node_id` trong cơ sở dữ liệu MongoDB thông qua Mongoose. Khi phát hiện thông số vượt ngưỡng, máy chủ sẽ điều khiển trực tiếp module SIM 4G A7680C ở lớp vật lý thông qua thư viện serialport để trích số điện thoại khách

hàng tương ứng, tự động khởi tạo cuộc gọi thoại và phát tin nhắn SMS cứu hộ khẩn cấp. Đồng thời, khối sử dụng Socket.io để đẩy dữ liệu và các trạng thái sự cố mạng tức thời xuống trình duyệt, cũng như nhận lệnh thay đổi ngưỡng từ người dùng để cập nhật vĩnh viễn vào Database.

- Khối xử lý dữ liệu: Chịu trách nhiệm điều phối các hoạt động của ứng dụng Web chạy tại trình duyệt bằng ngôn ngữ lập trình JavaScript. Nhiệm vụ chính của khối là thiết lập kết nối Socket.io Client để duy trì kênh giao tiếp song công thời gian thực với máy chủ Node.js. Khi nhận được các gói tin được bóc tách từ chuỗi JSON, khối tiến hành phân loại dữ liệu theo hai luồng rõ rệt: luồng dữ liệu cảm biến và luồng sự kiện cấu trúc mạng Mesh. Khối xử lý này sở hữu kiến trúc hướng dữ liệu linh hoạt, có khả năng khởi tạo giao diện động: tự động tạo thêm các phân vùng giám sát và các thực thể biểu đồ mới ngay khi phát hiện có node_id mới tham gia vào mạng lưới. Ngoài ra, khối đảm nhận việc liên tục cập nhật mảng dữ liệu trượt vào các đối tượng biểu đồ của thư viện Chart.js và điều hướng trạng thái logic của giao diện dựa trên các thông điệp định tuyến nhận được từ máy chủ.

- Khối hiển thị: Tầng giao diện trực quan trực tiếp tương tác với người dùng, được xây dựng từ ngôn ngữ định dạng HTML và bộ mã trang trí CSS, tối ưu hóa theo cấu trúc lưới khoa học giúp quan sát đa điểm nhiều nhà kho cùng lúc. Khối này đảm nhận việc chuyển đổi các kết quả xử lý từ JavaScript thành các thành phần đồ họa sinh động bao gồm: biểu đồ đường thể hiện xu hướng biến thiên liên tục, các thẻ chỉ số trực quan hiển thị giá trị tức thời, và đặc biệt là thanh trạng thái an toàn hệ thống thông minh (lập tức chuyển sang màu đỏ kèm thông điệp cảnh báo khi nhận sự kiện mất nút hạ tầng như Relay A sập nguồn, và tự động chuyển sang màu xanh khi mạng Mesh kích hoạt thành công cơ chế Self-Healing kết nối thẳng về Root Node). Khối hiển thị cũng cung cấp một bảng điều khiển cấu hình động tích hợp các ô nhập liệu, cho phép người quản trị tùy chỉnh ngưỡng an toàn cho từng nút thiết bị và đồng bộ trực tiếp xuống cơ sở dữ liệu của máy chủ trung tâm.

6. KẾT QUẢ THỰC HIỆN

6.1 Kết quả thực hiện hệ thống

Khi Leaf Node hoạt động nó sẽ scan để chọn ra Parent Node có RSSI mạnh nhất.

```
I (26162) LEAF_NODE: Scan xong: KHÔNG thấy Relay A/B  
I (28762) LEAF_NODE: Chọn parent: SSID="", BSSID=88:57:21:b3:56:f5, ch=8, RSSI=-49
```

Hình 6.1: Leaf Node chọn Parent Node

Dựa trên kết quả log từ Serial Monitor tại Hình 6.1, có thể thấy quá trình khởi tạo và gia nhập mạng của Leaf Node diễn ra hoàn toàn tự động. Node thực hiện quét các tín hiệu Beacon từ các Relay Node lân cận và so sánh chỉ số RSSI. Trong trường hợp này, Node đã chọn Parent có địa chỉ MAC 88:57:21:b3:56:f5 với mức tín hiệu -49 dBm, đây là mức tín hiệu rất tốt theo thang đo tại Bảng 2.1, đảm bảo truyền dẫn dữ liệu ổn định không bị mất gói.

Sau đó nó sẽ tiến hành thu thập dữ liệu từ các cảm biến xung quanh.

```
I (43892) LEAF_NODE: Light raw=4095, Vout=3.30 V  
I (44052) LEAF_NODE: Sent to ROOT 30:ae:a4:77:10:2d: {"node_id":"Leaf_01","role":"leaf","temp":29,"humi":79,"light_v":3.2999999523162842,"light_raw":4095,"motion":1}
```

Hình 6.2: Leaf Node gửi dữ liệu

Gói tin dữ liệu tại Hình 6.2 cho thấy cấu trúc chuỗi JSON được đóng gói chặt chẽ bao gồm các trường thông tin: node_id, temp, humi, lux và motion. Việc sử dụng định dạng JSON giúp dữ liệu có tính linh hoạt cao, dễ dàng bóc tách tại Root Node và mở rộng thêm các loại cảm biến khác trong tương lai mà không cần thay đổi cấu trúc lõi của hệ thống.

Và hiển thị lên màn hình OLED tại hiện trường giúp người dùng có thể theo dõi trực tiếp.



Hình 6.3: Dữ liệu hiển thị trên OLED

Khi 2 Relay Node hoạt động nó sẽ tìm Root Node để kết nối và sau đó nó sẽ phát softAP để tìm kiếm kết nối với Leaf Node ở gần nó nhất.

Relay Node A khởi động:

```
I (1114) RELAY_NODE_A: Mesh started
I (1114) RELAY_NODE_A: RELAY STA MAC : 00:70:07:7e:6f:bc
I (1114) RELAY_NODE_A: RELAY BSSID : 00:70:07:7e:6f:bd (Mesh SoftAP)
```

Hình 6.4: Địa chỉ STA và SoftAP của Relay Node A

Relay Node A kết nối với Root Node.

```
I (5694) RELAY_NODE_A: Parent 30:ae:a4:77:10:2d, layer=2, is_root=NO
I (5694) mesh: <MESH_NWK_ROOT_ADDR>node, layer=2, root_addr:30:ae:a4:77:10:2d, conflict_roots.num:0<>
I (5704) RELAY_NODE_A: Parent RSSI: -39 dBm
I (5714) RELAY_NODE_A: Root MAC: 30:ae:a4:77:10:2d
```

Hình 6.5: Relay Node A đã kết nối được với Root Node

Relay Node B khởi động:

```
I (1104) RELAY_NODE_B: Mesh started
I (1104) RELAY_NODE_B: RELAY STA MAC : 88:57:21:b3:56:f4
I (1114) RELAY_NODE_B: RELAY BSSID : 88:57:21:b3:56:f5 (Mesh SoftAP)
```

Hình 6.6: Địa chỉ STA và SoftAP của Relay Node B

Relay Node B kết nối với Root Node.

```
I (5704) RELAY_NODE_B: Parent 30:ae:a4:77:10:2d, layer=2, is_root=NO
I (5714) RELAY_NODE_B: Parent RSSI: -47 dBm
I (5724) RELAY_NODE_B: Root MAC: 30:ae:a4:77:10:2d
```

Hình 6.7: Relay Node B đã kết nối được với Root Node

Khi Root Node khởi động nó cho các Child Node kết nối với nó và nhận dữ liệu.

```
I (1102) ROOT_NODE: Mesh started (ROOT)
I (1102) ROOT_NODE: ROOT STA MAC : 30:ae:a4:77:10:2c
I (1102) ROOT_NODE: ROOT BSSID : 30:ae:a4:77:10:2d (Mesh SoftAP)
```

Hình 6.8: Địa chỉ STA và SoftAP của Root Node

Child Node kết nối với Root Node.

```
I (610412) ROOT_NODE: Mạng Mesh nhận lại nút / Tải kết nối thành công: +1 nút. Tổng=2
I (610422) ROOT_NODE: Node con mới kết nối trực tiếp: 00:70:07:7e:6f:bc, aid=1
I (638052) wifi:new:<6,0>, old:<6,0>, ap:<6,0>, sta:<6,0>, prof:1, snd_ch_cfg:0x0
I (638052) wifi:station: 88:57:21:b3:56:f4 join, AID=2, bgn, 20
I (638052) ROOT_NODE: Mạng Mesh nhận lại nút / Tải kết nối thành công: +1 nút. Tổng=3
I (638062) ROOT_NODE: Node con mới kết nối trực tiếp: 88:57:21:b3:56:f4, aid=2
```

Hình 6.9: Các Child Node đã kết nối được đến Root Node

Dựa trên các dòng log tại Hình 6.4 đến 6.9, ta có thể phân tích sâu về cơ chế định danh của mạng ESP-MESH. Mỗi Node khi khởi động sẽ tự tạo ra một SoftAP với BSSID được dẫn xuất từ địa chỉ MAC vật lý của chip ESP32. Ví dụ, Relay Node A có địa chỉ MAC kết thúc bằng :6f:bc và SoftAP kết thúc bằng :6f:bd. Việc chênh lệch 1 đơn vị trong địa chỉ MAC là đặc trưng của chip ESP32 khi chạy đồng thời hai chế độ STA và AP. Root Node đóng vai trò là 'Master' quản lý bảng định tuyến, cho phép các Child Node gia nhập thông qua quá trình bắt tay (handshake) và cấp phát AID (Association ID) như đã thấy trong log: AID=1, AID=2.

Root Node nhận dữ liệu từ Leaf Node thông qua Relay Node và gửi dữ liệu lên server MQTT.

```
I (387392) ROOT_NODE: Nhận 91 Bytes từ nút con [94:54:c5:e7:60:c8]: {"node_id":"Leaf_01","role":"leaf","temp":30,"humi":84,"lux":50.833332061767578,"motion":1}
I (392482) ROOT_NODE: Nhận 91 Bytes từ nút con [94:54:c5:e7:60:c8]: {"node_id":"Leaf_01","role":"leaf","temp":30,"humi":84,"lux":50.833332061767578,"motion":1}
I (397672) ROOT_NODE: Nhận 91 Bytes từ nút con [94:54:c5:e7:60:c8]: {"node_id":"Leaf_01","role":"leaf","temp":30,"humi":82,"lux":50.833332061767578,"motion":1}
I (402852) ROOT_NODE: Nhận 91 Bytes từ nút con [94:54:c5:e7:60:c8]: {"node_id":"Leaf_01","role":"leaf","temp":30,"humi":83,"lux":51.666664123535156,"motion":1}
I (408032) ROOT_NODE: Nhận 91 Bytes từ nút con [94:54:c5:e7:60:c8]: {"node_id":"Leaf_01","role":"leaf","temp":30,"humi":82,"lux":50.833332061767578,"motion":1}
I (413212) ROOT_NODE: Nhận 91 Bytes từ nút con [94:54:c5:e7:60:c8]: {"node_id":"Leaf_01","role":"leaf","temp":30,"humi":83,"lux":50.833332061767578,"motion":1}
I (418402) ROOT_NODE: Nhận 91 Bytes từ nút con [94:54:c5:e7:60:c8]: {"node_id":"Leaf_01","role":"leaf","temp":30,"humi":84,"lux":50.833332061767578,"motion":1}
I (423572) ROOT_NODE: Nhận 91 Bytes từ nút con [94:54:c5:e7:60:c8]: {"node_id":"Leaf_01","role":"leaf","temp":30,"humi":83,"lux":50.833332061767578,"motion":1}
I (428752) ROOT_NODE: Nhận 91 Bytes từ nút con [94:54:c5:e7:60:c8]: {"node_id":"Leaf_01","role":"leaf","temp":30,"humi":83,"lux":50.833332061767578,"motion":1}
```


Hình 6.10: Root Node đã nhận được dữ liệu từ Leaf Node

Server MQTT nhận được dữ liệu từ Root Node publish lên topic đã đăng ký.

```
C:\Program Files\mosquitto>mosquitto_sub -h 192.168.1.9 -p 1883 -t warehouse/sensors/data
{"node_id":"Leaf_01","role":"leaf","temp":30,"humi":80,"lux":50.833332061767578,"motion":0}
{"node_id":"Leaf_01","role":"leaf","temp":30,"humi":81,"lux":50.833332061767578,"motion":0}
{"node_id":"Leaf_01","role":"leaf","temp":30,"humi":81,"lux":50.833332061767578,"motion":0}
{"node_id":"Leaf_01","role":"leaf","temp":30,"humi":80,"lux":50.833332061767578,"motion":0}
{"node_id":"Leaf_01","role":"leaf","temp":30,"humi":80,"lux":50.833332061767578,"motion":0}
{"node_id":"Leaf_01","role":"leaf","temp":30,"humi":80,"lux":50.833332061767578,"motion":0}
```

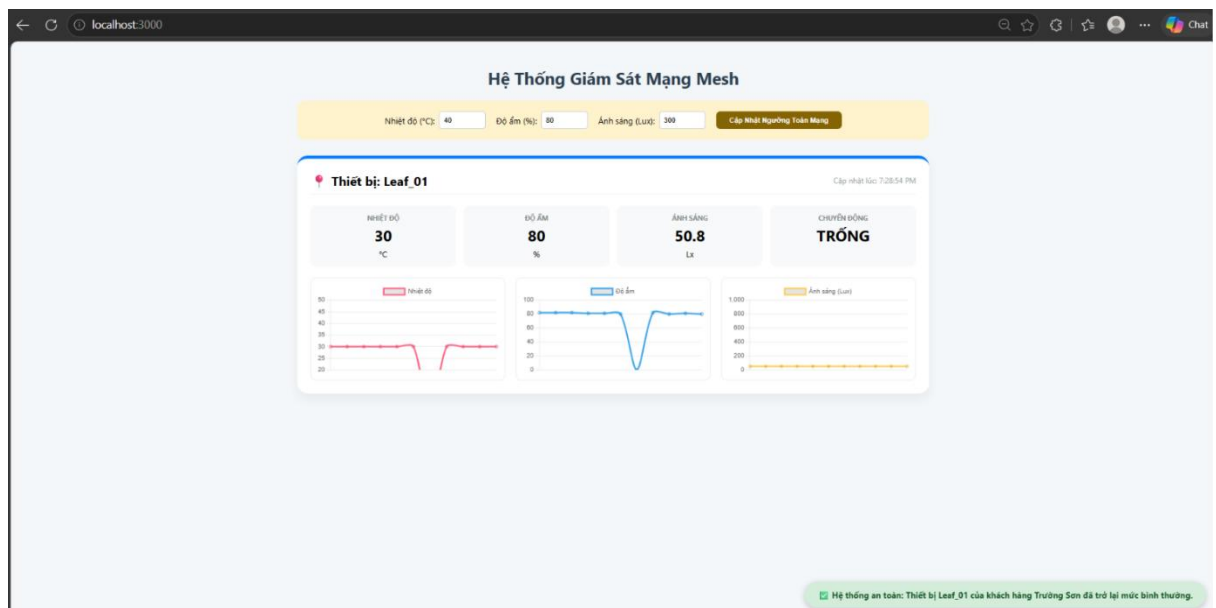
Hình 6.11: Topic chứa dữ liệu cảm biến

```
C:\Program Files\mosquitto>mosquitto_sub -h 192.168.1.9 -p 1883 -t warehouse/mesh/status
{"status":"ALERT","msg":"CẢNH BÁO: Phát hiện nút hạ tầng mất kết nối."}
{"status":"OK","msg":" Hệ thống tự sửa lỗi (Self-Healing) thành công!"}
```

Hình 6.12: Topic chứa báo cáo trạng thái mạng

Hệ thống phân chia dữ liệu trên MQTT Broker thành hai đường dẫn chính là warehouse/sensors/data để truyền dữ liệu cảm biến và warehouse/mesh/status để báo cáo trạng thái mạng. Thay vì tạo đường dẫn riêng cho từng thiết bị, tất cả các nút đều gửi chung về hai đường dẫn này. Máy chủ trung tâm sẽ phân biệt từng vị trí bằng một mã node_id duy nhất nằm bên trong chuỗi JSON gửi kèm. Cách thiết kế này giúp máy chủ dễ dàng phân loại dữ liệu, đối chiếu với cơ sở dữ liệu và quản lý chính xác hàng trăm điểm đo khác nhau trong nhà kho mà không lo bị trùng lặp.

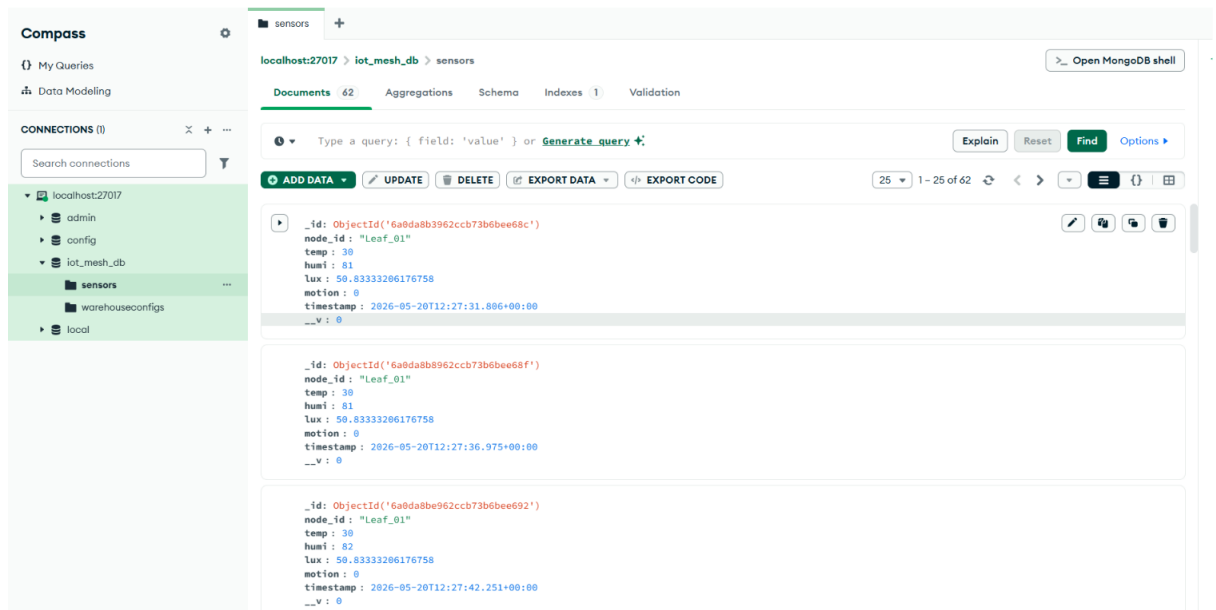
Khi website được khởi động nó sẽ thực hiện nhận dữ liệu từ hệ thống mạng Mesh gửi lên thông qua giao thức MQTT.



Hình 6.13: Dashboard của hệ thống

Giao diện Dashboard tại Hình 6.13 được tối ưu hóa cho trải nghiệm người dùng với các thẻ chỉ số hiển thị giá trị tức thời và biểu đồ đường để theo dõi xu hướng. Khi có chuyển động được phát hiện bởi cảm biến PIR, trạng thái trên Web sẽ lập tức chuyển từ 'TRỐNG' sang 'CÓ NGƯỜI' với độ trễ phản hồi gần như bằng không nhờ vào công nghệ Web Socket (Socket.io) duy trì kết nối song công giữa Server và trình duyệt.

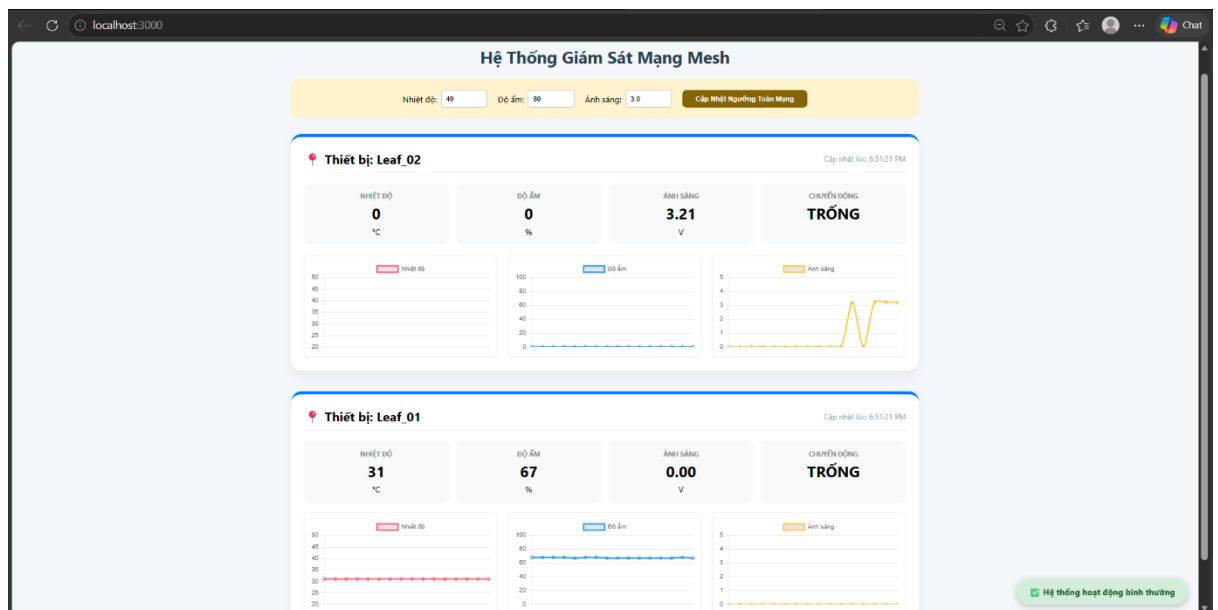
Toàn bộ dữ liệu sau khi được thu thập từ các nút mạng và truyền lên máy chủ sẽ được tiến hành lưu trữ bền vững vào cơ sở dữ liệu MongoDB theo thời gian thực. Cơ chế lưu trữ tự động và liên tục này không chỉ giúp bảo vệ dữ liệu khỏi nguy cơ mất mát khi hệ thống gặp sự cố mạng, mà còn tạo nền tảng vững chắc cho việc truy xuất lịch sử, phục vụ công tác phân tích và vẽ biểu đồ xu hướng biến thiên dài hạn trên giao diện Dashboard.



Hình 6.14: Dữ liệu lưu trữ tại Mongo DB

Cấu trúc dữ liệu lưu trữ trong MongoDB (Hình 6.14) được thiết kế theo mô hình phi lược đồ đặc trưng của NoSQL. Mỗi Document bao gồm các trường: node_id, temp, humi, lux, motion và quan trọng nhất là timestamp. Trường timestamp được Server tự động chèn vào thời điểm nhận được gói tin thông qua Socket.io, giúp đảm bảo tính chính xác về mặt thời gian khi thực hiện truy vấn lịch sử hoặc vẽ biểu đồ xu hướng dài hạn trên Dashboard.

Hệ thống Dashboard được thiết kế với khả năng tự động khởi tạo giao diện động ngay khi có thiết bị mới gia nhập mạng Mesh. Ngay khi nhận được gói tin từ một Leaf Node mới qua Socket.io, khối xử lý JavaScript sẽ tự động nhận diện node_id và khởi tạo thêm các khu vực hiển thị tương ứng. Cơ chế này cho phép người dùng giám sát chính xác số lượng nút mạng thực tế trên Website một cách trực quan mà không cần can thiệp thủ công hay tải lại trang. Việc cập nhật thời gian thực này đảm bảo tính linh hoạt và khả năng mở rộng quy mô dễ dàng cho hệ thống giám sát nhà kho.

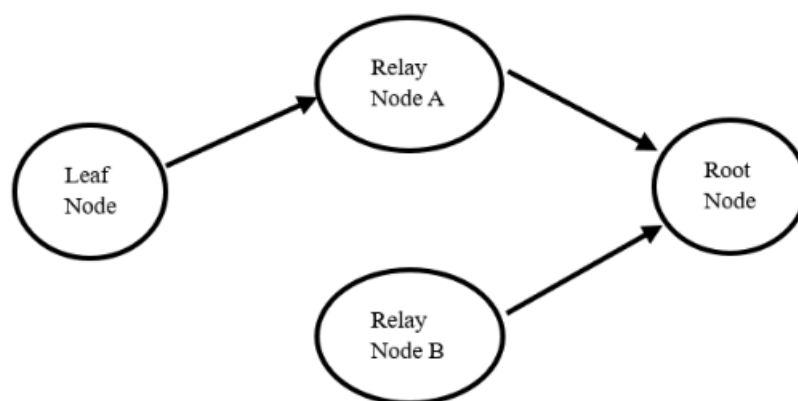


Hình 6.15: 2 Leaf Node gửi dữ liệu

Quá trình khi mạng Mesh bị rớt 1 node và thực hiện chức năng tự phục hồi cho mạng. Cấu hình để hệ thống thực hiện được quy trình từ Leaf Node đến Root Node, Root Node sẽ được cấu hình chỉ kết nối được 2 Child Node như vậy thì sẽ đảm bảo đường đi từ Leaf Node -> Relay Node -> Root Node.

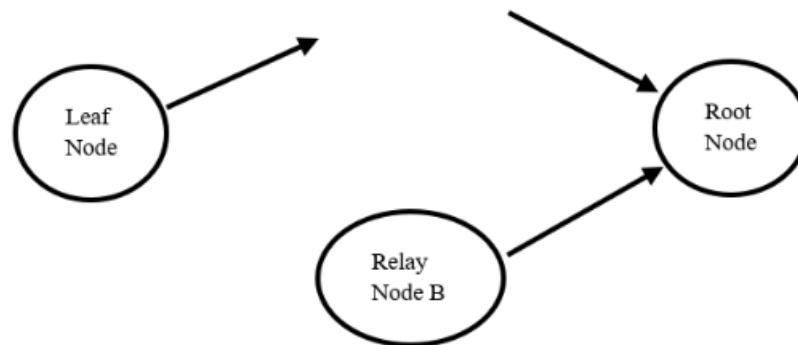
- Chức năng tự phục hồi của mạng Mesh

Khi hệ thống hoạt động bình thường. Thể hiện qua hình minh họa sau:

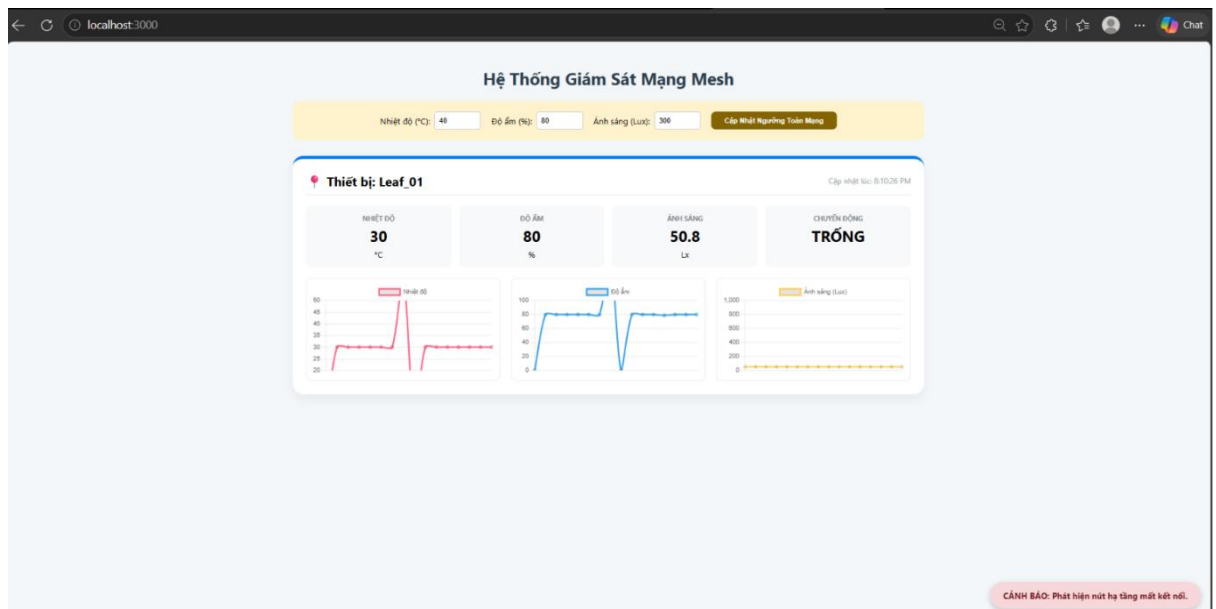


Hình 6.16: Hệ thống hoạt động bình thường

Khi hệ thống gặp sự cố mất kết nối với 1 node trong hệ thống. Thể hiện qua hình minh họa và dashboard cũng hiển thị cho người dùng biết.

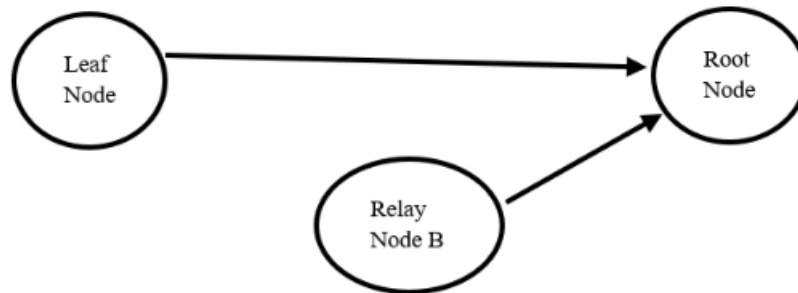


Hình 6.17: Hệ thống gặp sự cố

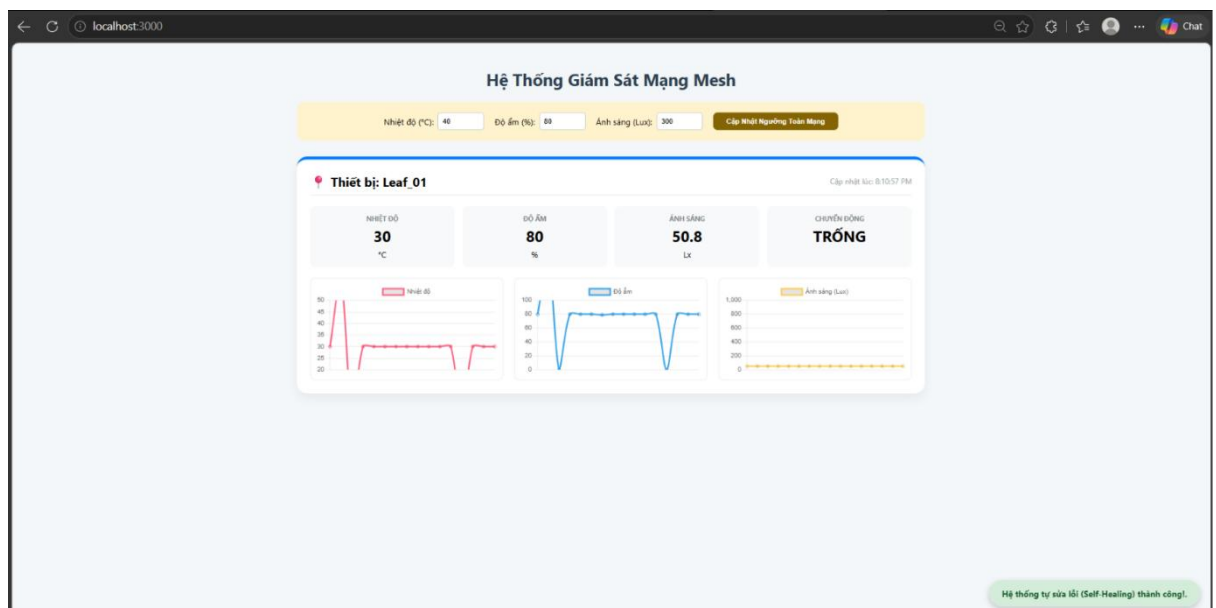


Hình 6.18: Dashboard hiển thị hệ thống gặp sự cố mất node

Sau đó hệ thống thực hiện chức năng tự phục hồi. Vì các thiết bị để khoảng cách không quá xa nên là Leaf Node thực hiện chọn Parent Node để kết nối. Nó nhận thấy rằng RSSI mạnh nhất trong thời điểm Leaf Node scan và cũng như khoảng cách đến Root Node ngắn nhất chính là Leaf Node tự kết nối đến Root Node.



Hình 6.19: Chức năng tự phục hồi của ESP-Mesh



Hình 6.20: Dashboard hiển thị hệ thống đã thực hiện chức năng tự phục hồi

```

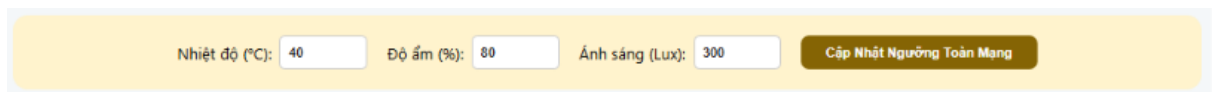
W (576432) ROOT_NODE: Node con đã ngắt kết nối: 88:57:21:b3:56:f4, aid=2
I (576432) mesh: [scan]new scanning time:300ms, beacon interval:100ms
W (576432) ROOT_NODE: Bảng định tuyến sụt giảm nút! Giảm 1 nút. Tổng số nút còn lại=3
I (577092) ROOT_NODE: Nhận 112 Bytes từ nút con [94:54:c5:e7:60:c8]: {"node_id":"Leaf_01":82,"light_v":3.2999999523162842,"light_raw":4095,"motion":1}
I (582282) ROOT_NODE: Nhận 112 Bytes từ nút con [94:54:c5:e7:60:c8]: {"node_id":"Leaf_01":82,"light_v":3.2999999523162842,"light_raw":4095,"motion":1}
I (586452) mesh: [scan]new scanning time:600ms, beacon interval:300ms
I (587472) ROOT_NODE: Nhận 112 Bytes từ nút con [94:54:c5:e7:60:c8]: {"node_id":"Leaf_01":82,"light_v":3.2999999523162842,"light_raw":4095,"motion":1}
I (592672) ROOT_NODE: Nhận 112 Bytes từ nút con [94:54:c5:e7:60:c8]: {"node_id":"Leaf_01":82,"light_v":3.2999999523162842,"light_raw":4095,"motion":1}
I (597842) ROOT_NODE: Nhận 112 Bytes từ nút con [94:54:c5:e7:60:c8]: {"node_id":"Leaf_01":82,"light_v":3.2999999523162842,"light_raw":4095,"motion":1}
W (598502) ROOT_NODE: Bảng định tuyến sụt giảm nút! Giảm 1 nút. Tổng số nút còn lại=2
I (605852) wifi:new:<6,0>, old:<6,0>, ap:<6,0>, sta:<6,0>, prof:1, snd_ch_cfg:0x0
I (605852) wifi:station: 94:54:c5:e7:60:c8 join, AID=2, bgn, 20
I (605852) ROOT_NODE: Mạng Mesh nhận lại nút / Tái kết nối thành công: +1 nút. Tổng=3
I (605862) ROOT_NODE: Node con mới kết nối trực tiếp: 94:54:c5:e7:60:c8, aid=2
  
```

Hình 6.21: Quá trình mạng Mesh thực hiện chức năng tự phục hồi

Khi Root Node hoạt động nó đã kết nối được với 2 thiết bị Relay Node A và B có địa chỉ MAC lần lượt là 88:57:21:b3:56:f4 và 00:70:07:7e:6f:bc, Leaf Node kết nối với Relay Node A (88:57:21:b3:56:f4), như vậy hệ thống có 4 node đang hoạt động. Nhưng khi 1 trong 2 thiết bị mất kết nối như trong hình là thiết bị có địa chỉ MAC 88:57:21:b3:56:f4 bị mất kết nối. Thì cũng lúc đó Leaf Node cũng sẽ không thể gửi dữ liệu lên trên nữa và bị ngắt kết nối. Sau đó hệ thống dò thì thấy thiết bị có địa chỉ MAC 94:54:c5:e7:60:c8 (Leaf Node) vẫn còn hoạt động thì lập tức được Root Node kết nối thay cho Relay Node A vừa bị mất kết nối để đảm bảo quá trình truyền dữ liệu từ Leaf Node lên Root Node. Đây là chức năng tự phục hồi của ESP-Mesh.

- Chức năng chỉnh sửa ngưỡng an toàn từ người dùng trực tiếp trên Dashboard và chức năng cảnh báo nguy hiểm đến người dùng.

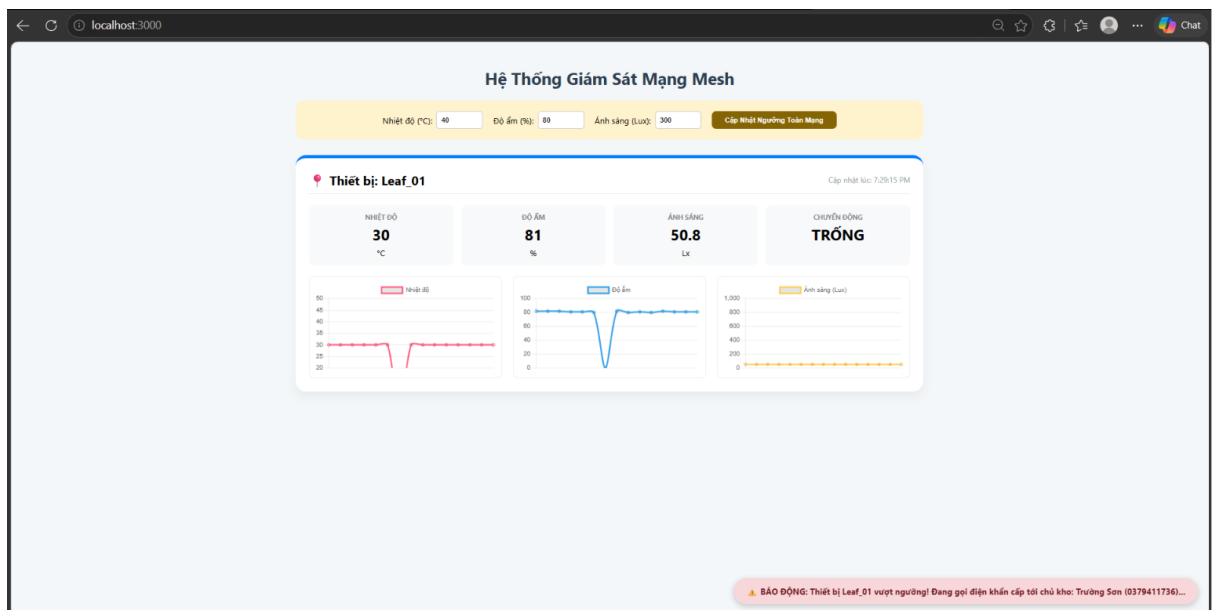
Chức năng chỉnh sửa ngưỡng an toàn cho hệ thống theo ý định của người dùng. Cho phép người dùng có thể điều chỉnh các ngưỡng như của nhiệt độ, độ ẩm và ánh sáng.



The image shows a user interface for setting thresholds. It features three input fields: 'Nhiệt độ (°C):' with the value '40', 'Độ ẩm (%):' with the value '80', and 'Ánh sáng (Lux):' with the value '300'. To the right of these fields is a button labeled 'Cập Nhật Ngưỡng Toàn Mạng'.

Hình 6.22: Chức năng điều chỉnh ngưỡng

Khi hệ thống phát hiện vượt ngưỡng an toàn cho phép. Nó sẽ hiển thị cảnh báo thông số nào đang vượt quá ngưỡng cho phép và bắt đầu thực hiện cảnh báo.



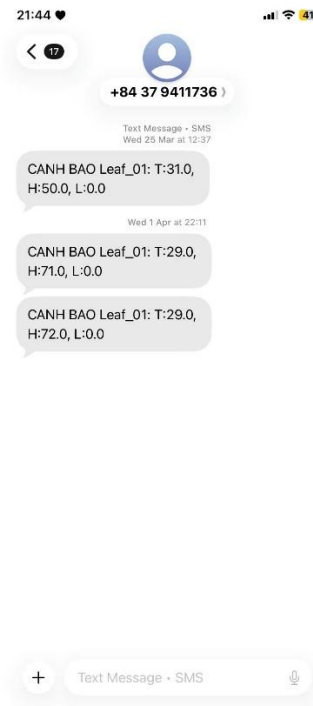
Hình 6.23: Hệ thống phát hiện vượt ngưỡng

Hệ thống khi nhận được cảnh báo ngưỡng thay đổi và cũng như nhận được cảnh báo khi mà hệ thống phát hiện nguy hiểm vượt quá ngưỡng an toàn nó sẽ thực hiện hàm cảnh báo đến người dùng.

```
[KÍCH HOẠT SIM 4G] Đang thực hiện cuộc gọi khẩn cấp tới: 0379411736
[SIM 4G] Đã phát tin nhắn SMS tới số 0379411736 thành công!
[Dữ liệu từ Leaf_01]: T=30°C, H=80%, L=50.83333206176758 Lux
[SIM 4G] Đã gác máy số 0379411736, tiến hành gửi tin nhắn SMS...
[Dữ liệu từ Leaf_01]: T=30°C, H=80%, L=50.83333206176758 Lux
[SIM 4G] Đã phát tin nhắn SMS tới số 0379411736 thành công!
```

Hình 6.24: Thực hiện hàm cảnh báo

Tin nhắn SMS nội dung cảnh báo về trạng thái của các cảm biến và cuộc gọi sẽ được gửi số điện thoại mà người sở hữu node đăng ký trong database.

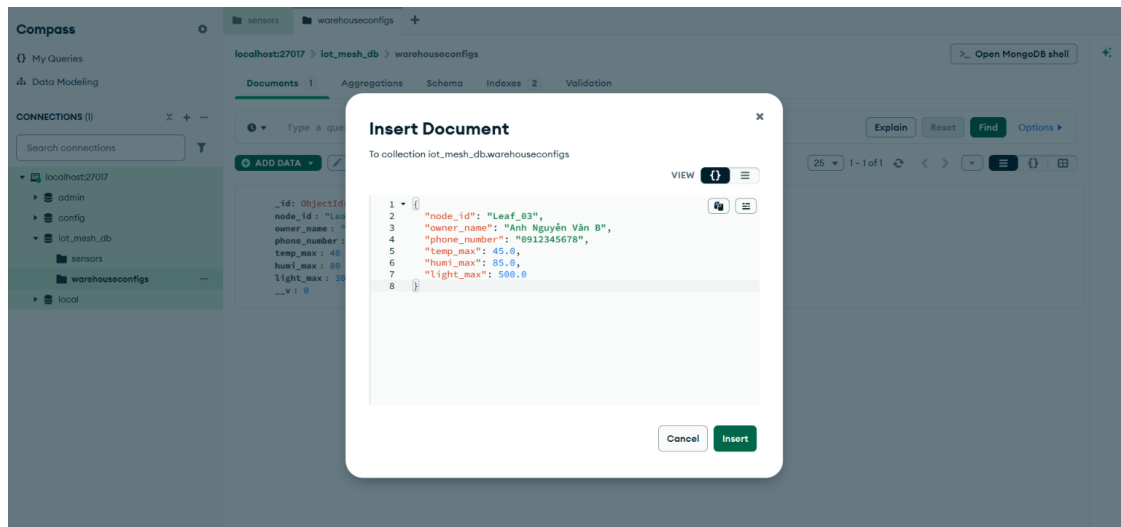


Hình 6.25: Nhận cảnh báo bằng tin nhắn SMS ở điện thoại

Hiệu quả của module A7680C được thể hiện rõ qua tốc độ đáp ứng. Khi giá trị nhiệt độ đạt 30 độ C trong khi ngưỡng cài đặt là 20 độ C, hệ thống kích hoạt hàm cảnh báo. Nội dung tin nhắn SMS được định dạng ngắn gọn nhưng đầy đủ thông số: CANH BAO Leaf_01: T:31.0, H:50.0, L:0.0. Điều này cực kỳ hữu ích trong môi trường công nghiệp, giúp người vận hành biết chính xác Node nào đang gặp sự cố và thông số cụ thể là bao nhiêu để có phương án xử lý kịp thời.

- Tối ưu hóa chi phí cho hệ thống, chỉ dùng 1 module SIM đặt ở server để thực hiện cảnh báo đến các khách hàng đăng ký dữ liệu lưu trong database.

Trước khi hệ thống đi vào vận hành, thông tin của từng khách hàng sẽ được lưu sẵn vào cơ sở dữ liệu MongoDB thành một bảng danh bạ. Mỗi bản ghi sẽ bao gồm: mã thiết bị node_id, tên chủ kho, số điện thoại nhận thông báo và các ngưỡng giới hạn an toàn của nhiệt độ, độ ẩm, ánh sáng. Việc lưu trữ trước này giúp máy chủ nhận diện chính xác mỗi thiết bị đang lắp ở đâu và thuộc quyền sở hữu của ai.



Hình 6.26: Đăng ký thông tin khách hàng lên database

Khi nhận dữ liệu, máy chủ sử dụng hàm `WarehouseConfig.findOne` để tra cứu mã `node_id` trong MongoDB, từ đó tự động lấy ra đúng số điện thoại và ngưỡng cảnh báo của riêng nhà kho đó. Đồng thời, biến đối tượng `alerting_status` sẽ lưu trữ riêng trạng thái báo động của từng nút, giúp máy chủ biết rõ thiết bị nào đã được gọi điện hay chưa để kích hoạt cảnh báo độc lập cho nhiều kho cùng lúc mà không bị chồng chéo dữ liệu. Nhờ cơ chế quản lý trạng thái bằng phần mềm này, hệ thống chỉ cần sử dụng duy nhất một module SIM đặt tại máy chủ trung tâm để điều phối và thực hiện các cuộc gọi cứu hộ tuần tự cho toàn bộ mạng lưới. Giải pháp này giúp tối ưu hóa tối đa chi phí đầu tư phần cứng mà vẫn đảm bảo tính chính xác và kịp thời.

6.2 Kết quả kiểm tra hệ thống

6.2.1 Đo đạc số liệu cảm biến DHT11

Sử dụng kết quả đo cảm biến DHT11 trên website và đối chứng kết quả thu được với thiết bị đo nhiệt độ độ ẩm Electronic Hygrometer.



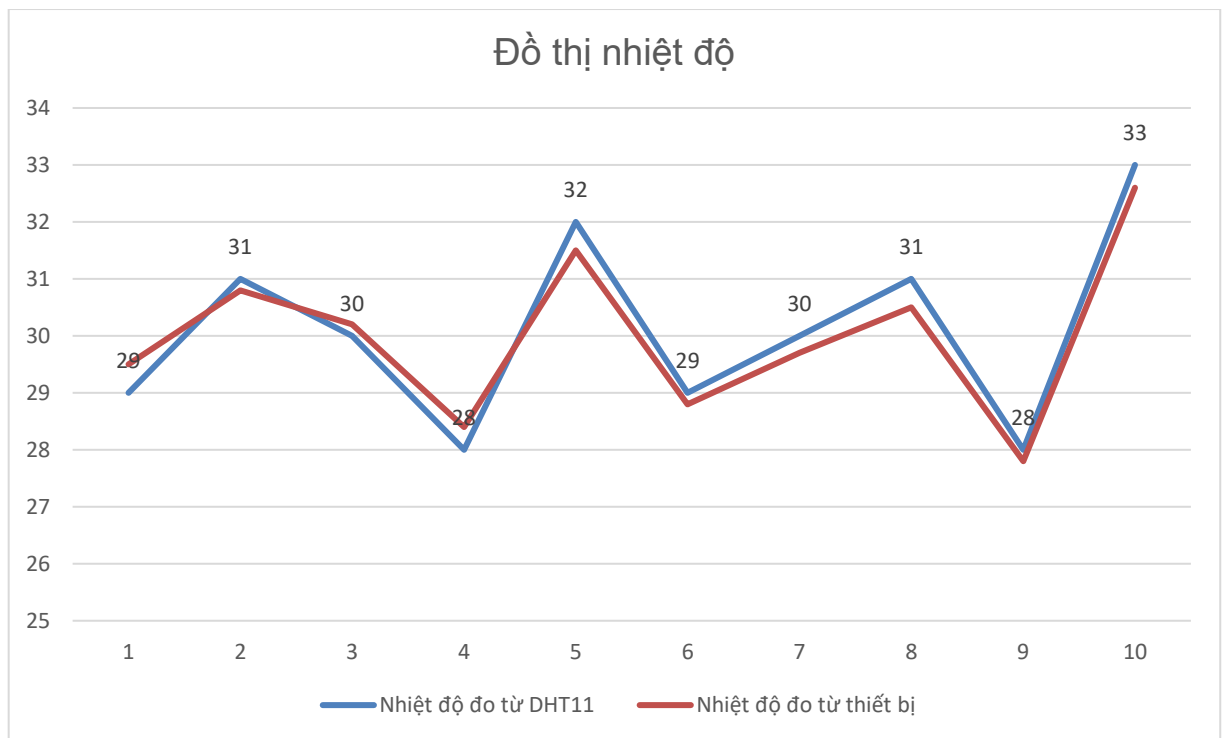
Hình 6.27: Thiết bị đo Electronic Hygrometer

Thực hiện đo kết quả 10 lần trong ngày ta có bảng số liệu sau:

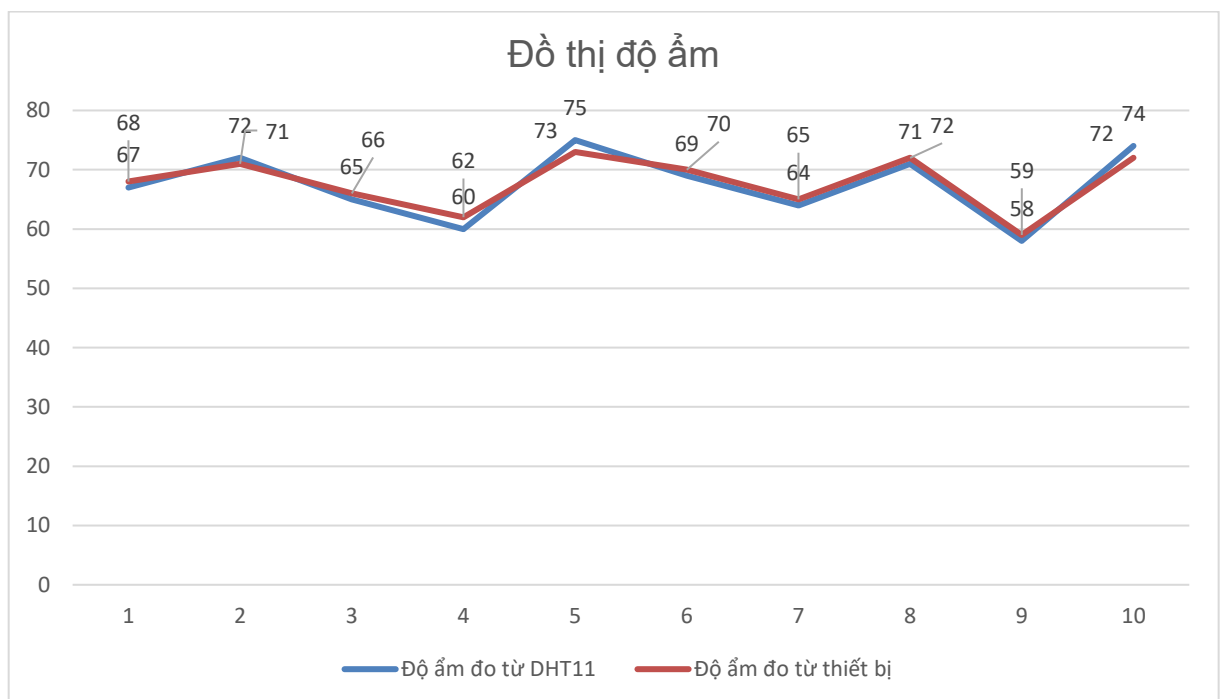
Bảng 6.1: Giá trị đo từ cảm biến DHT11 so với thiết bị

Số lần đo	Nhiệt độ đo từ DHT11	Nhiệt độ đo từ thiết bị	Độ ẩm đo từ DHT11	Độ ẩm đo từ thiết bị
1	29	29.5	67	68
2	31	30.8	72	71
3	30	30.2	65	66
4	28	28.4	60	62
5	32	31.5	75	73
6	29	28.8	69	70
7	30	29.7	64	65
8	31	30.5	71	72
9	28	27.8	58	59
10	33	32.6	74	72

Từ bảng ta có các đồ thị sau:



Hình 6.28: Kết quả so sánh nhiệt độ giữa cảm biến và thiết bị



Hình 6.29: Kết quả so sánh độ ẩm giữa cảm biến và thiết bị

Dựa trên các bảng số liệu đo đạc thực tế giữa hệ thống tự thiết kế và các thiết bị đo tham chiếu tiêu chuẩn, việc xác định độ lệch là bước tối quan trọng để khẳng định độ tin cậy của đồ án. Để định lượng chính xác hiệu năng của các cảm biến

trong môi trường nhà kho, ta tiến hành tính toán sai số tương đối (%) cho từng lần đo.

Công thức tính sai số được thiết lập dựa trên nguyên lý so sánh giá trị thực nghiệm và giá trị tham chiếu chuẩn như sau:

$$\text{Sai số} = \frac{|\text{Giá trị đo từ DHT11} - \text{Giá trị đo từ thiết bị}|}{\text{Giá trị đo từ thiết bị}} 100\%$$

Sau khi thực hiện quy trình đối chứng dữ liệu giữa cảm biến DHT11 và thiết bị đo chuẩn Electronic Hygrometer, các giá trị chênh lệch đã được tính toán định lượng bằng công thức sai số tương đối. Việc tổng hợp các kết quả này vào bảng thống kê dưới đây giúp chúng ta có cái nhìn trực quan về độ ổn định của hệ thống qua từng mốc thời gian đo đặc khác nhau. Qua đó, người quản lý có thể đánh giá được mức độ tin cậy của các thông số nhiệt độ và độ ẩm đang được hiển thị trên Dashboard trước khi đưa ra các quyết định điều vận nhà kho. Số liệu chi tiết về sai số phần trăm của 10 mẫu đo được trình bày cụ thể như sau:

Bảng 6.2: Giá trị sai số của giá trị đo từ cảm biến DHT11 so với thiết bị

Số lần đo	Nhiệt độ	Độ ẩm
1	1.69%	1.47%
2	0.65%	1.41%
3	0.66%	1.52%
4	1.41%	3.23%
5	1.59%	2.74%
6	0.69%	1.43%
7	1.01%	1.54%
8	1.64%	1.39%
9	0.72%	1.69%
10	1.23%	2.78%

Đánh giá kết quả:

Mặc dù sai số nhiệt độ dưới 3% và độ ẩm dưới 5% là kết quả khả quan, nhưng qua thực nghiệm em nhận thấy có một hiện tượng 'trễ nhiệt' khoảng 2-3 phút so với thiết bị chuẩn khi nhiệt độ môi trường thay đổi đột ngột. Điều này là do vỏ nhựa của DHT11 có độ dày nhất định, làm chậm quá trình trao đổi nhiệt của cảm biến bên trong. Tuy nhiên, trong môi trường nhà kho với diện tích lớn, nhiệt độ thường biến thiên chậm nên đặc tính này không gây ảnh hưởng đáng kể đến độ tin cậy của hệ thống giám sát.

6.2.2 Đo đặc số liệu cảm biến BH1750 GY-302

Sử dụng kết quả đo cảm biến BH1750 GY-302 trên website và đối chứng kết quả thu được với thiết bị đo ánh sáng Luxmeter.



Hình 6.30: Thiết bị đo Luxmeter

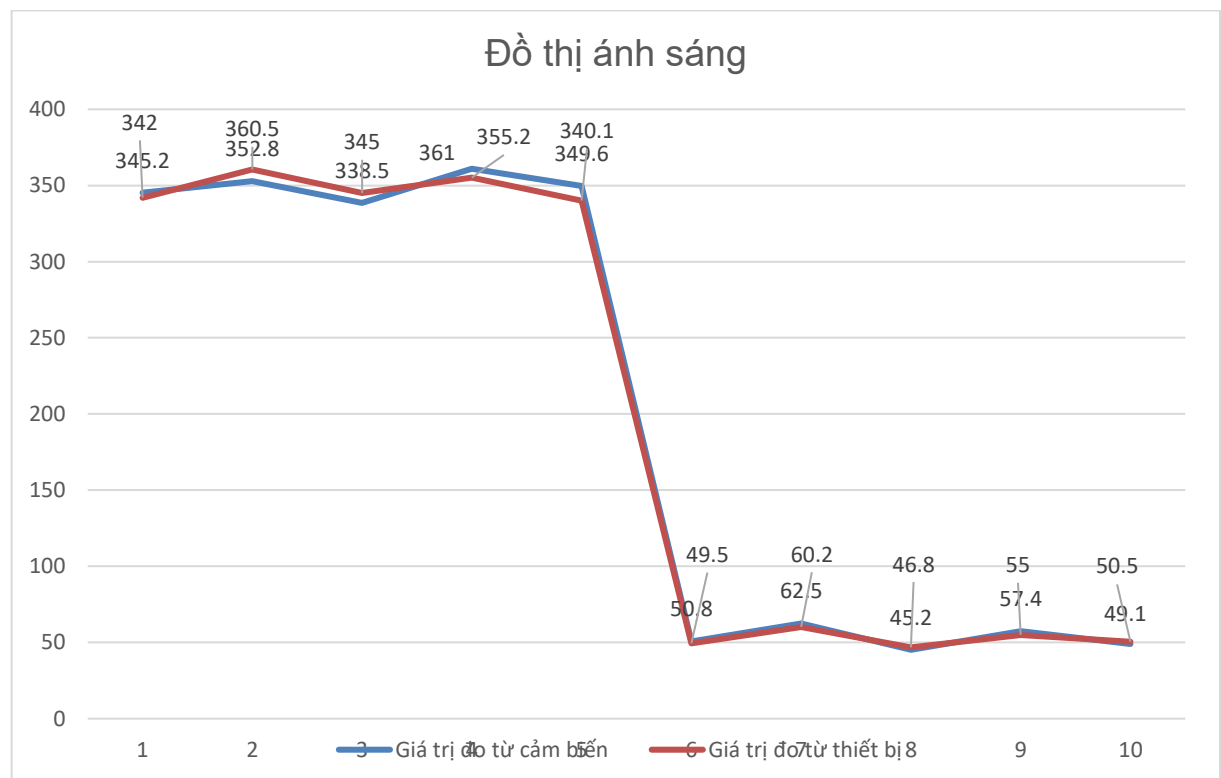
Thực hiện đo kết quả 10 lần trong ngày ta có bảng số liệu sau:

Bảng 6.3: Giá trị đo từ cảm biến BH1750 GY-302 so với thiết bị

Số lần đo	Nhiệt độ đo từ BH1750 GY-302	Nhiệt độ đo từ thiết bị
1	345.2	342
2	352.8	360.5
3	338.5	345
4	361	355.2
5	349.6	340.1
6	50.8	49.5

7	62.5	60.2
8	45.2	46.8
9	57.4	55
10	49.1	50.5

Từ bảng ta có đồ thị sau:



Hình 6.31: Kết quả so sánh giá trị ánh sáng giữa cảm biến và thiết bị

Dựa trên các bảng số liệu đo đạc thực tế giữa hệ thống tự thiết kế và các thiết bị đo tham chiếu tiêu chuẩn, việc xác định độ lệch là bước tối quan trọng để khẳng định độ tin cậy của đồ án. Để định lượng chính xác hiệu năng của các cảm biến trong môi trường nhà kho, ta tiến hành tính toán sai số tương đối (%) cho từng lần đo.

Công thức tính sai số được thiết lập dựa trên nguyên lý so sánh giá trị thực nghiệm và giá trị tham chiếu chuẩn như sau:

$$Sai\ số = \frac{|Giá\ trị\ từ\ cảm\ biến - Giá\ trị\ đo\ từ\ thiết\ bị|}{Giá\ trị\ đo\ từ\ thiết\ bị} 100\%$$

Sau khi thực hiện quy trình đối chứng dữ liệu giữa cảm biến BH1750 GY-302 và thiết bị đo chuẩn Luxmeter, các giá trị chênh lệch đã được tính toán định lượng bằng công thức sai số tương đối. Việc tổng hợp các kết quả này vào bảng thống kê dưới đây giúp chúng ta có cái nhìn trực quan về độ ổn định của hệ thống qua từng mốc thời gian đo đặc khác nhau. Qua đó, người quản lý có thể đánh giá được mức độ tin cậy của các thông số ánh sáng đang được hiển thị trên Dashboard trước khi đưa ra các quyết định điều vận nhà kho. Số liệu chi tiết về sai số phần trăm của 10 mẫu đo được trình bày cụ thể như sau:

Bảng 6.4: Giá trị sai số của giá trị đo từ cảm biến BH1750 GY-302 so với thiết bị

Số lần đo	Giá trị ánh sáng
1	0.94%
2	2.14%
3	1.88%
4	1.63%
5	2.79%
6	2.63%
7	3.82%
8	3.42%
9	4.36%
10	2.77%

Đánh giá kết quả:

Mặc dù sai số cường độ ánh sáng của BH1750 ổn định dưới 5% so với thiết bị chuẩn, thực nghiệm cho thấy cảm biến dễ bị dao động nhẹ khi thay đổi góc đón sáng hoặc dưới tần số quét của đèn điện. Điều này do mắt nhận phẳng của BH1750 thu sáng kém toàn diện hơn thấu kính vòm của máy chuẩn. Tuy nhiên, trong môi trường nhà kho có nguồn sáng cố định và biến thiên chậm, đặc tính này không ảnh hưởng đến độ tin cậy của hệ thống.

7. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

7.1 Kết luận

Sau thời gian tìm hiểu và thực hiện đề tài, em đã tiếp thu được thêm nhiều kiến thức về thiết kế và lập trình hệ thống nhúng và các phần mềm liên quan để thực hiện cho đề tài. Em cũng hiểu rõ hơn về công nghệ Mesh, cách thiết kế một website và lưu trữ dữ liệu.

Hệ thống cơ bản đã đạt được những mục tiêu đặt ra ban đầu ở phần tổng quan như sau:

- Thu thập dữ liệu từ môi trường và hệ thống cảnh báo nguy hiểm đến người dùng.
- Truyền thành công dữ liệu từ các node qua lại với nhau và lên server với tỉ lệ gói tin bị mất thấp.
- Thành công trong việc tạo được website quản lí trực quan để quan sát thay đổi khi cần thiết.

Tuy nhiên, hệ thống vẫn còn một số nhược điểm, những thiếu sót chưa được khắc phục như:

- Độ chính xác của các cảm biến trong hệ thống chi phí thị trường thấp vẫn chưa cao.
- Khả năng truyền tín hiệu của hệ thống bị ảnh hưởng nếu mạng internet chậm chạp.

7.2 Hướng phát triển

Bởi vì thời gian thực hiện đề tài có giới hạn nên vẫn còn nhiều phương án, thiết kế mà em chưa thể tìm hiểu thêm và tối ưu hóa cho hệ thống. Những phương án đó nhằm bổ sung chức năng và cải thiện giải thuật. Cụ thể các hướng phát triển có thể cải thiện thêm cho hệ thống.

- Cải thiện tối ưu về thuật toán giúp hệ thống đáp ứng được nhanh hơn có cảnh báo vượt ngưỡng an toàn.

- Cải thiện thêm về độ chính xác của dữ liệu.
- Cải thiện về giao diện cho website.
- Cải thiện về bảo mật khi truyền dữ liệu.
- Tích hợp thêm nhiều module, cảm biến để có thể thu thập đầy đủ các yếu tố khác ảnh hưởng trong môi trường nhà kho.

8. TÀI LIỆU THAM KHẢO

- [1] Espressif Systems (Shanghai) Co., Ltd, “ESP-WIFI-MESH”, docs.espressif.com
- [2] Espressif Systems (Shanghai) Co., Ltd, “ESP-WIFI-MESH”, docs.espressif.com
- [3] CircuitLabs, “Chapter 46: ESP32 WIFI Mesh Networking”, circuitlabs.net
- [4] Espressif Systems (Shanghai) Co., Ltd, “ESP-WIFI-MESH”, docs.espressif.com
- [5] CTS Group, “RSSI: Chỉ số cường độ tín hiệu thu – Cách tính suy hao và công suất thu Wifi”, ctsgroup.com.vn
- [6] Espressif Systems (Shanghai) CO., LTD, “ESP-MESH Programming Guide”, docs.espressif.com
- [7] OASIS (Organization for the Advancement of Structured Information Standards), “MQTT: The Standard for IoT Messaging”, mqtt.org
- [8] Eclipse Foundation, “Eclipse Mosquitto”, mosquitto.org
- [9] SIMCom, “A76XX Series_AT_Command_Manual_V1.06”, aliyuncs.com
- [10] Refsnes Data, “HTML Styles – CSS”, www.w3schools.com/html/html_css.asp
- [11] Refsnes Data, “JavaScript Tutorial” , www.w3schools.com/js/default.asp
- [12] Espressif Systems, *ESP32 Technical Reference Manual*,
https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf
- [13] OpenJS Foundation, *Node.js API Reference Documentation*, <https://nodejs.org/api/>
- [14] MongoDB Inc., *MongoDB Manual v7.0*, <https://www.mongodb.com/docs/manual/>
- [15] Socket.io, *Socket.io Server-Client Communication Documentation*,
<https://socket.io/docs/v4/>

- [16] Tạp chí Khoa học và Công nghệ - Đại học Đà Nẵng, *Thiết kế hệ thống IoT giám sát môi trường ứng dụng truyền thông 4G LTE*, <https://jst-ud.vn/>
- [17] IoTZone, *Cách xây dựng ESP32 Mesh / ESP8266 Mesh Network để gửi dữ liệu cảm biến*, <https://iotzone.vn/esp32/esp32-mesh-network/>
- [18] Giljr, *esp32_mesh*, https://github.com/giljr/esp32_mesh

9. PHỤ LỤC

- Leaf node

```
static bool find_best_parent(mesh_parent_t *out)
10. {
11.     wifi_scan_config_t sc = { .ssid=0, .bssid=0, .channel=0,
    .show_hidden=true };
12.     ESP_ERROR_CHECK(esp_wifi_scan_start(&sc, true));
13.     uint16_t ap_num = 0;
14.     ESP_ERROR_CHECK(esp_wifi_scan_get_ap_num(&ap_num));
15.     if (ap_num == 0) {
16.         ESP_LOGI(TAG, "Scan xong: KHÔNG thấy AP nào");
17.         return false;
18.     }
19.     wifi_ap_record_t *recs = (wifi_ap_record_t*)calloc(ap_num,
    sizeof(wifi_ap_record_t));
20.     if (!recs) {
21.         ESP_LOGE(TAG, "calloc ap records fail");
22.         return false;
23.     }
24.     uint16_t n = ap_num;
25.     esp_err_t e = esp_wifi_scan_get_ap_records(&n, recs);
26.     if (e != ESP_OK) {
27.         ESP_LOGE(TAG, "esp_wifi_scan_get_ap_records err: %s",
    esp_err_to_name(e));
28.         free(recs);
29.         return false;
30.     }
31.     bool needA = bssid_is_nonzero(RELAY_A_BSSID);
32.     bool needB = (!ONLY_USE_RELAY_A) &&
    bssid_is_nonzero(RELAY_B_BSSID);
33.
34.     bool seenA = false, seenB = false;
35.     wifi_ap_record_t recA = {0}, recB = {0};
36.
37.     for (int i = 0; i < n; ++i) {
```

```

38.     if (needA && !memcmp(recs[i].bssid, RELAY_A_BSSID, 6)) { recA =
    recs[i]; seenA = true; }
39.     if (needB && !memcmp(recs[i].bssid, RELAY_B_BSSID, 6)) { recB =
    recs[i]; seenB = true; }
40. }
41. if (!seenA && !seenB) {
42.     ESP_LOGI(TAG, "Scan xong: KHÔNG thấy Relay A/B");
43.     free(recs);
44.     return false;
45. }
46. const wifi_ap_record_t *best = NULL;
47. if (seenA && seenB) best = (recA.rssi >= recB.rssi) ? &recA :
    &recB;
48. else if (seenA)     best = &recA;
49. else                best = &recB;
50.
51. memset(out, 0, sizeof(*out));
52. memcpy(out->bssid, best->bssid, 6);
53. strncpy(out->ssid, (const char*)best->ssid, sizeof(out->ssid));
54. out->channel = best->primary;
55. out->rssi     = best->rssi;
56. ESP_LOGI(TAG, "Chọn parent: SSID=\"%s\", BSSID=" MACSTR ", ch=%u,
    RSSI=%d",
57.         out->ssid, MAC2STR(out->bssid), out->channel, out->rssi);
58. free(recs);
59. return true;
60.}

```

Hàm `find_best_parent`: Hàm quét Wi-Fi theo chế độ blocking để thu danh sách AP, sau đó chỉ lọc hai BSSID đã định danh trước (Relay A/B). Nếu tìm thấy, hàm so sánh cường độ tín hiệu (RSSI) và chọn parent có chất lượng tốt hơn, đồng thời điền thông tin ứng viên (BSSID, SSID, kênh, RSSI) vào cấu trúc đầu ra và trả về true; nếu không thấy A/B hoặc xảy ra lỗi khi quét/lấy bản ghi, hàm giải phóng bộ nhớ tạm và trả về false. Cách tiếp cận này bảo đảm leaf chỉ bám đúng các relay mong muốn, tránh join nhầm AP cùng SSID nhưng khác BSSID.

```

61. static void mesh_event_handler(void *arg, esp_event_base_t base,
    int32_t id, void *event_data)
62. {
63.     switch (id) {
64.     case MESH_EVENT_PARENT_CONNECTED: {
65.         g_mesh_connected = true;

```

```

66.     mesh_event_connected_t *connected = (mesh_event_connected_t
*)event_data;
67.     esp_mesh_get_parent_bssid(&g_parent_bssid);
68.
69.     ESP_LOGI(TAG, "PARENT_CONNECTED: " MACSTR ", layer:%d",
70.             MAC2STR(g_parent_bssid.addr), connected->self_layer);
71.     wifi_ap_record_t ap_info;
72.     if (esp_wifi_sta_get_ap_info(&ap_info) == ESP_OK) {
73.         ESP_LOGI(TAG, "Parent RSSI: %d dBm", ap_info.rssi);
74.     }
75.     try_set_bandwidth();
76.     log_path();
77.     break;
78. }
79. case MESH_EVENT_PARENT_DISCONNECTED:
80.     g_mesh_connected = false;
81.     ESP_LOGW(TAG, "PARENT_DISCONNECTED -> reselect");
82.     schedule_reselect_parent();
83.     break;
84. case MESH_EVENT_NO_PARENT_FOUND: {
85.     mesh_event_no_parent_found_t *e =
(mesh_event_no_parent_found_t*)event_data;
86.     ESP_LOGW(TAG, "NO_PARENT_FOUND scan=%d -> reselect", e-
>scan_times);
87.     schedule_reselect_parent();
88.     break;
89. }
90. case MESH_EVENT_LAYER_CHANGE: {
91.     mesh_event_layer_change_t *e =
(mesh_event_layer_change_t*)event_data;
92.     ESP_LOGI(TAG, "Layer -> %d", e->new_layer);
93.     break;
94. }
95. case MESH_EVENT_ROOT_ADDRESS: {
96.     const mesh_event_root_address_t *e = (const
mesh_event_root_address_t *)event_data;
97.     memcpy(g_root_addr.addr, e->addr, 6);
98.     g_root_addr_ok = true;
99.     ESP_LOGI(TAG, "Root MAC: " MACSTR, MAC2STR(g_root_addr.addr));
100.    break;
101.    }
102.    default:
103.        break;
104.    }
105. }

```

Hàm `mesh_event_handler`: Bộ xử lý sự kiện trung tâm của Mesh, cập nhật trạng thái kết nối và tuyến đường. Khi `PARENT_CONNECTED`, hàm đặt cờ

g_mesh_connected, ghi nhận BSSID của parent, log RSSI và đường đi; khi PARENT_DISCONNECTED hoặc NO_PARENT_FOUND, hàm xóa cờ kết nối và kích hoạt quy trình chọn lại parent (schedule_reselect_parent()). Với MESH_EVENT_ROOT_ADDRESS, hàm lưu MAC của root vào g_root_addr và đặt cờ g_root_addr_ok để các tác vụ gửi dữ liệu biết điểm đích. Ngoài ra, hàm log thay đổi layer để phục vụ giám sát.

```

106.     static void send_sensor_task(void *arg)
107.     {
108.         while (!g_mesh_connected || !g_root_addr_ok)
109.             vTaskDelay(pdMS_TO_TICKS(300));
110.         ESP_LOGI(TAG, "TX ready: layer=%u, root=" MACSTR,
111.             esp_mesh_get_layer(), MAC2STR(g_root_addr.addr));
112.         for (;;) {
113.             struct dht11_reading dht = DHT11_read();
114.             int temp = 0, hum = 0;
115.             if (dht.status == DHT11_OK) { temp = dht.temperature; hum
116. = dht.humidity; }
117.             else { ESP_LOGW(TAG, "DHT11 read
118. error"); }
119.             int motion = gpio_get_level(PIR_PIN);
120.             float lux = bh1750_read_lux();
121.             if (lux < 0) {
122.                 ESP_LOGE(TAG, "BH1750 read error!");
123.                 lux = 0.0f;
124.             }
125.             ESP_LOGI(TAG, "Light BH1750: %.2f Lux", lux);
126.             char line[24];
127.             ssd1306_clear(&oled);
128.             ssd1306_display_text(&oled, 0, "Node: Leaf_01", false);
129.             snprintf(line, sizeof(line), "Temp:%dC", temp);
130.             ssd1306_display_text(&oled, 2, line, false);
131.             snprintf(line, sizeof(line), "Humi:%d%%", hum);
132.             ssd1306_display_text(&oled, 3, line, false);
133.             snprintf(line, sizeof(line), "Light: %.1fLx", lux);
134.             ssd1306_display_text(&oled, 4, line, false);
135.             snprintf(line, sizeof(line), "Motion:%s", motion ? "YES"
136. : "NO");
137.             ssd1306_display_text(&oled, 5, line, false);
138.             cJSON *root = cJSON_CreateObject();

```

```

136.         cJSON_AddStringToObject(root, "node_id", "Leaf_01");
137.         cJSON_AddStringToObject(root, "role", "leaf");
138.         cJSON_AddNumberToObject(root, "temp", temp);
139.         cJSON_AddNumberToObject(root, "humi", humi);
140.         cJSON_AddNumberToObject(root, "lux", lux);
141.         cJSON_AddNumberToObject(root, "motion", motion);
142.
143.         char *json_str = cJSON_PrintUnformatted(root);
144.         size_t len = json_str ? strlen(json_str, sizeof(tx_buf)
- 1) : 0;
145.         if (json_str && len > 0) {
146.             memcpy(tx_buf, json_str, len);
147.             tx_buf[len] = '\0';
148.         } else {
149.             const char *fallback = "{\"err\":\"json\"}";
150.             len = strlen(fallback);
151.             memcpy(tx_buf, fallback, len + 1);
152.         }
153.
154.         data.data = tx_buf;
155.         data.size = len;
156.         data.proto = MESH_PROTO_BIN;
157.         data.tos = MESH_TOS_P2P;
158.
159.         mesh_addr_t dest = {0};
160.         memcpy(dest.addr, g_root_addr.addr, 6);
161.         esp_err_t err = esp_mesh_send(&dest, &data,
MESH_DATA_P2P, NULL, 0);
162.         if (err == ESP_OK) ESP_LOGI(TAG, "Sent to ROOT " MACSTR
": %s", MAC2STR(dest.addr), (char*)data.data);
163.         else ESP_LOGE(TAG, "Mesh send failed: %s
(0x%x)", esp_err_to_name(err), err);
164.
165.         if (json_str) free(json_str);
166.         cJSON_Delete(root);
167.         vTaskDelay(pdMS_TO_TICKS(5000));
168.     }
169. }

```

Hàm `send_sensor_task` : chịu trách nhiệm thu thập và truyền dữ liệu. Nó chỉ bắt đầu sau khi đã có kết nối mesh và biết địa chỉ root (`g_mesh_connected && g_root_addr_ok`), sau đó lặp: đọc cảm biến DHT11/PIR/BH1750, hiển thị nhanh lên OLED, đóng gói payload JSON (`node_id`, `role`, `temp`, `humi`, `lux`, `motion`), cấu hình `mesh_data_t` và gửi P2P tới root bằng `esp_mesh_send()`, ghi log thành công/thất bại rồi nghỉ 5 giây. Thiết kế này tách biệt rõ phần truyền thông (mesh) và phần cảm biến/hiển thị, giúp dễ bảo trì và theo dõi lỗi.

- Relay node

```
170.     static void mesh_event_handler(void *arg, esp_event_base_t base,
171.         int32_t id, void *event_data) {
172.         switch (id) {
173.             case MESH_EVENT_STARTED: {
174.                 ESP_LOGI(TAG, "Mesh started");
175.                 try_set_bw20();
176.                 break;
177.             }
178.             case MESH_EVENT_PARENT_CONNECTED: {
179.                 g_mesh_connected = true;
180.                 mesh_event_connected_t *e =
181.                     (mesh_event_connected_t*)event_data;
182.                 esp_mesh_get_parent_bssid(&g_parent_bssid);
183.                 ESP_LOGI(TAG, "Parent " MACSTR ", layer=%d,
184.                     is_root=%s",
185.                         MAC2STR(g_parent_bssid.addr), e->self_layer,
186.                         esp_mesh_is_root() ? "YES" : "NO");
187.                 wifi_ap_record_t ap_info;
188.                 if (esp_wifi_sta_get_ap_info(&ap_info) == ESP_OK) {
189.                     ESP_LOGI(TAG, "Parent RSSI: %d dBm",
190.                         ap_info.rssi);
191.                 }
192.                 try_set_bw20();
193.                 break;
194.             }
195.             case MESH_EVENT_PARENT_DISCONNECTED: {
196.                 g_mesh_connected = false;
197.                 ESP_LOGW(TAG, "Parent disconnected");
198.                 break;
199.             }
200.             case MESH_EVENT_ROUTING_TABLE_ADD: {
201.                 mesh_event_routing_table_change_t *e =
202.                     (mesh_event_routing_table_change_t*)event_data;
203.                 ESP_LOGI(TAG, "+RT: %d, total=%d", e->rt_size_change,
204.                     e->rt_size_new);
205.                 break;
206.             }
207.             case MESH_EVENT_ROUTING_TABLE_REMOVE: {
208.                 mesh_event_routing_table_change_t *e =
209.                     (mesh_event_routing_table_change_t*)event_data;
210.                 ESP_LOGI(TAG, "-RT: %d, total=%d", e->rt_size_change,
211.                     e->rt_size_new);
212.                 break;
213.             }
214.             case MESH_EVENT_ROOT_ADDRESS: {
215.                 const mesh_event_root_address_t *ra = (const
216.                     mesh_event_root_address_t*)event_data;
```



```

208.             memcpy(g_root_addr.addr, ra->addr, 6);
209.             g_have_root = true;
210.             ESP_LOGI(TAG, "Root MAC: " MACSTR,
MAC2STR(g_root_addr.addr));
211.             break;
212.         }
213.         default:
214.             break;
215.     }
216. }

```

Hàm `mesh_event_handler`: lắng nghe toàn bộ sự kiện ESP-MESH để cập nhật trạng thái topo: khi mesh start thì ép HT20; khi nối được parent thì bật `g_mesh_connected`, lưu BSSID của parent, in layer và RSSI; khi mất parent thì tắt cò và chờ tự tổ chức lại; khi nhận `ROOT_ADDRESS` thì lưu MAC root vào `g_root_addr` và bật `g_have_root`.

```

static void mesh_sniff_task(void *arg) {
    mesh_addr_t from;
    uint8_t rx_buf[256];
    mesh_data_t rx = {
        .data = rx_buf,
        .size = sizeof(rx_buf),
        .proto = MESH_PROTO_BIN,
        .tos = MESH_TOS_P2P
    };
    int flag = 0;
    for (;;) {
        rx.size = sizeof(rx_buf);
        esp_err_t err = esp_mesh_recv(&from, &rx, 1000 /
portTICK_PERIOD_MS, &flag, NULL, 0);
        if (err == ESP_OK) {
            size_t n = (rx.size < sizeof(rx_buf)) ? rx.size :
sizeof(rx_buf)-1;
            rx_buf[n] = '\0';
            ESP_LOGI(TAG, "RX (child=" MACSTR "): %s", MAC2STR(from.addr),
(char*)rx.data);
            flag = 0;
        }
    }
}

```

Hàm `mesh_sniff_task`: là hàm thu các gói dữ liệu trọng ESP-MESH nó chạy vòng lặp `esp_mesh_recv()` để nhận gói P2P nhị phân từ các Child node, chuẩn hóa chuỗi rồi in log kèm MAC nguồn và đẩy lên Root node.

- Root Node

```
static void mesh_recv_task(void *arg) {
    mesh_addr_t from;
    uint8_t rx_buf[512];
    mesh_data_t rx = { .data = rx_buf, .size = sizeof(rx_buf), .proto
= MESH_PROTO_BIN, .tos = MESH_TOS_DEF };
    int flag = 0;

    for(;;){
        rx.size = sizeof(rx_buf);
        esp_err_t err = esp_mesh_recv(&from, &rx, portMAX_DELAY,
&flag, NULL, 0);
        if (err == ESP_OK) {
            rx_buf[rx.size] = '\0';

            ESP_LOGI(TAG, "Nhận %u Bytes từ nút con [" MACSTR "]: %s",
(unsigned)rx.size, MAC2STR(from.addr), (char*)rx_buf);

            if (g_mqtt_connected && g_mqtt) {
                esp_mqtt_client_publish(g_mqtt, MQTT_DATA_TOPIC,
(const char*)rx.data, (int)rx.size, 0, 0);
            }
        }
    }
}
```

Hàm `mesh_recv_task` đóng vai trò chuyển tiếp dữ liệu từ mạng ESP-MESH lên máy chủ trung tâm. Tác vụ này liên tục lắng nghe và tiếp nhận các gói tin cảm biến từ các nút con thông qua hàm `esp_mesh_recv`. Ngay khi nhận được, Root Node không bóc tách dữ liệu mà lập tức gửi nguyên văn chuỗi JSON đó lên chủ đề cố định `warehouse/sensors/data` thông qua giao thức MQTT. Thiết kế này giúp giảm tải tính toán tối đa cho vi điều khiển, nhường toàn bộ logic so sánh ngưỡng, lưu trữ MongoDB và kích hoạt gọi điện cảnh báo qua module SIM cho máy chủ xử lý.

```
static void mesh_apply_config_open(void) {
    mesh_cfg_t cfg = MESH_INIT_CONFIG_DEFAULT();
    memcpy(cfg.mesh_id.addr, MESH_ID, 6);
    cfg.channel = ROUTER_CHANNEL;
    cfg.router.ssid_len = strlen(ROUTER_SSID);
    strncpy((char*)cfg.router.ssid, ROUTER_SSID,
sizeof(cfg.router.ssid));
}
```

```
    strncpy((char*)cfg.router.password, ROUTER_PASS,
sizeof(cfg.router.password));

    cfg.mesh_ap.max_connection      = 2;
    cfg.mesh_ap.nonmesh_max_connection = 0;
    memset(cfg.mesh_ap.password, 0, sizeof(cfg.mesh_ap.password));

    ESP_ERROR_CHECK(esp_mesh_set_config(&cfg));
    ESP_ERROR_CHECK(esp_mesh_set_ap_authmode(WIFI_AUTH_OPEN));
}
```

Hàm `mesh_apply_config_open` thực hiện thiết lập các tham số vận hành cho mạng ESP-MESH, bao gồm định danh mạng (Mesh ID), kênh truyền và thông tin đăng nhập vào bộ định tuyến (Router) để duy trì kết nối Internet. Để tối ưu hóa hiệu suất và bảo mật nội bộ, hệ thống giới hạn mỗi node tối đa 2 kết nối con và chặn hoàn toàn các thiết bị ngoại vi không thuộc cấu trúc Mesh. Đặc biệt, mạng được cấu hình ở chế độ xác thực mở (`WIFI_AUTH_OPEN`), cho phép các node gia nhập hệ thống nhanh chóng dựa trên định danh Mesh ID chung mà không cần mật khẩu Wi-Fi nội bộ, giúp đơn giản hóa quá trình kết nối và mở rộng mạng.